Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации

Ордена Трудового Красного Знамени

федеральное государственное бюджетное образовательное учреждение высшего образования

«МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

СВЯЗИ И ИНФОРМАТИКИ»

Отчет по лабораторным работам

по дисциплине

«Прикладные информационные системы»

Выполнил:

студент гр. БСТ2251

Лебедев М. В.

Проверил: Сергеев А. Е.

Москва 2026

Содержание

Лабораторная работа №1 4

1.Основы языка Python 4

2.Установка и запуск веб-фреймворка django 8

Лабораторная работа №2 17

1.Создание первой web-страницы с простым текстом. 17

2.Создание первого html-шаблона 20

3.Настройка обработки статичных файлов для django 26

Лабораторная работа №3 31

1.Создание первой модели данных и её регистрация в административном приложении django 31

2.Динамическое генерирование шаблона для вывода всех экземпляров этой модели 34

Лабораторная работа №4 39

1.Создание страницы определенной записи 39

2.Верстка обеих страниц в соответствии с макетом 41

Лабораторная работа №5 45

1. Создание формы и представления для нового поста 45

Лабораторная работа №6 53

1.Создание формы, шаблона и представления для регистрации 53

2.Создание формы, шаблона и представления для авторизации 57

Лабораторная работа №7 63

1.Изучение основ JavaScript, создание простейших функций и использование базовых операторов 63

2.Работа с элементами DOM с помощью JavaScript 66

Лабораторная работа №8 71

1.Изучение библиотеки jQuery, добавление подсветки для наведенного поста и эффекта для картинки-логотипа 71

2. 8

Лабораторная работа №9 71

1. Дальнейшее изучение библиотеки jQuery, добавление эффекта параллакса для главной страницы блога 75

Список литературы 81

Лабораторная работа №1

1. Основы языка Python

Создадим файл с расширением «.py» назвав его helloworld и через редактор кода запишем в него строку «print "Hello, world!"».

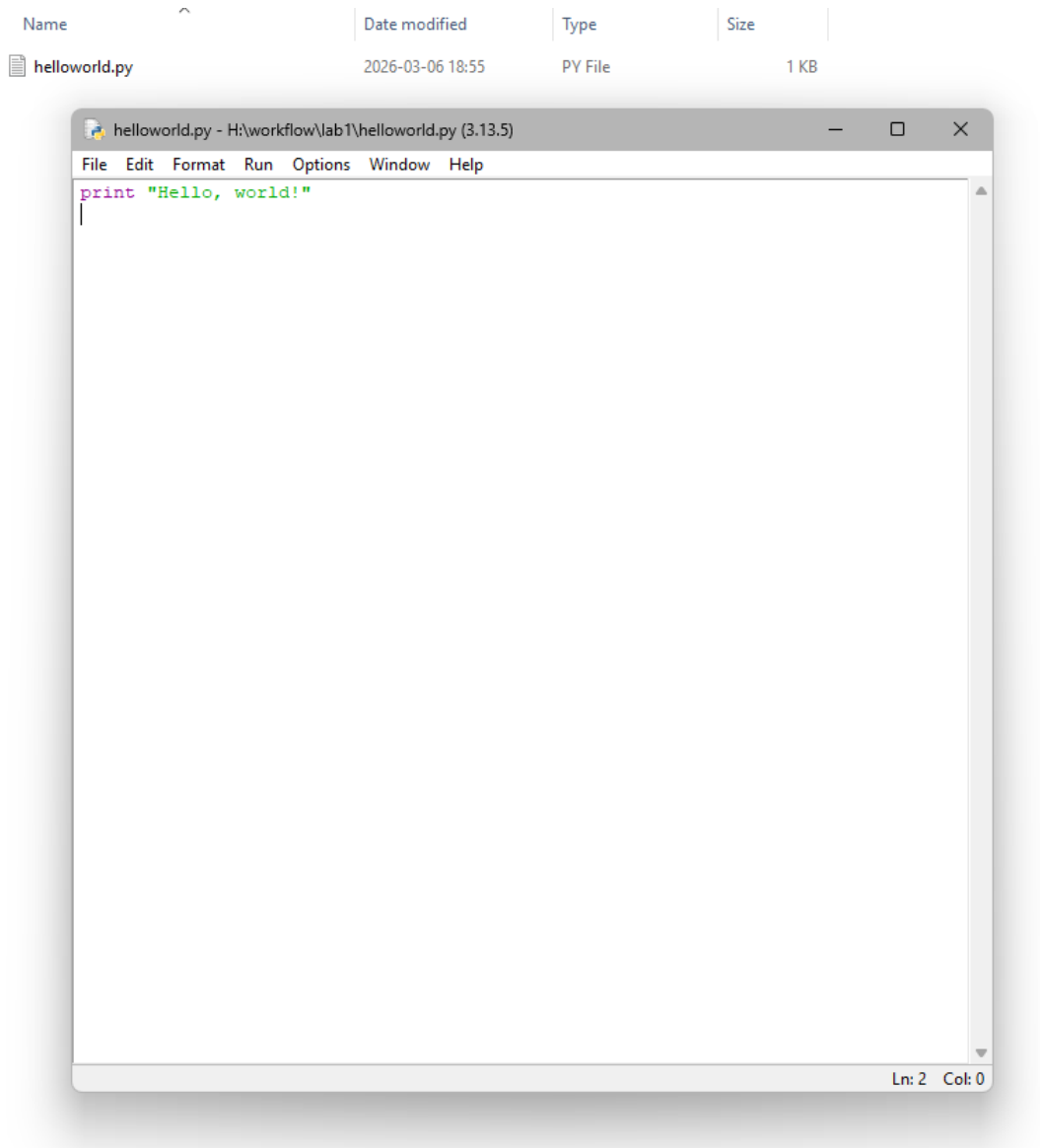

Рисунок 1 – Редактор кода Python IDLE

Выведем надпись в командной оболочке:

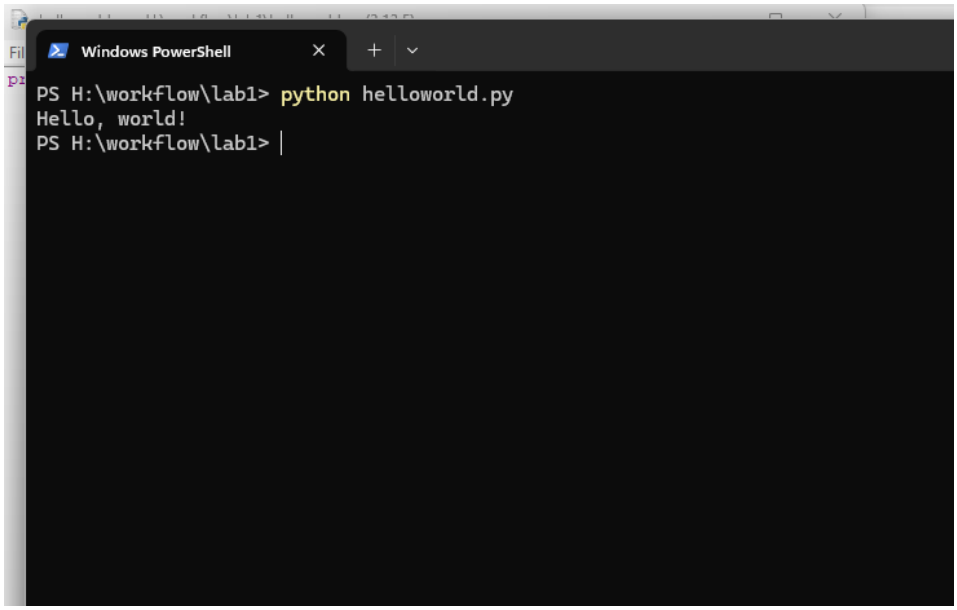

Рисунок 2 – Вывод данных на экран

Далее создадим список студентов

groupmates.py - H:\workflow\lab1\groupmates.py (3.13.5)

File Edit Format Run Options Window Help

```
groupmates = [
    {
        "name": "Алексей",
        "group": "БСТ0001",
        "age": 20,
        "marks": [3, 3, 4, 5, 4]
    },
    {
        "name": "Анна",
        "group": "БСТ0002",
        "age": 18,
        "marks": [3, 2, 3, 4, 3]
    },
    {
        "name": "Борис",
        "group": "БСТ0002",
        "age": 24,
        "marks": [3, 5, 4, 3, 5]
    },
    {
        "name": "Екатерина",
        "group": "БСТ0003",
        "age": 19,
        "marks": [5, 3, 5, 4, 5]
    }
    {
        "name": "Константин",
        "group": "БСТ0001",
        "age": 19,
        "marks": [5, 3, 3, 4, 5]
    }
]
```

Рисунок 3 – Список студентов

Добавим функцию print_students

groupmates.py - H:\workflow\lab1\groupmates.py (3.13.5)

File Edit Format Run Options Window Help

```
groupmates = [
    {
        "name": "Алексей",
        "group": "БСТ0001",
        "age": 20,
        "marks": [3, 3, 4, 5, 4]
    },
    {
        "name": "Анна",
        "group": "БСТ0002",
        "age": 18,
        "marks": [3, 2, 3, 4, 3]
    },
    {
        "name": "Борис",
        "group": "БСТ0002",
        "age": 24,
        "marks": [3, 5, 4, 3, 5]
    },
    {
        "name": "Екатерина",
        "group": "БСТ0003",
        "age": 19,
        "marks": [5, 3, 5, 4, 5]
    }
    {
        "name": "Константин",
        "group": "БСТ0001",
        "age": 19,
        "marks": [5, 3, 3, 4, 5]
    }
]

def print_students(students):
    print("Имя студента".ljust(15), "Группа".ljust(8), "Возраст".ljust(8), "Оценки".lj

    for student in students:
        print(
            student["name"].ljust(15),
            student["group"].ljust(8),
            str(student["age"]).ljust(8),
            str(student["marks"]).ljust(20)
        )
    print("\n")
```

Рисунок 4 – Функция print_students

Добавим функцию фильтрации студентов по средней оценке (так чтобы функция возвращала всех студентов выше заданного в параметрах функции среднего балла). Программный код представлен в листинге 1.

Листинг 1. Фильтрация студентов по средней оценке

```
# Фильтрует студентов по средней оценке

def filter_students_by_avg_mark(students, min_avg_mark):


    filtered_students = []  # создаем пустой массив для отфильтрованных
студентов


    for student in students:

        # считаем среднюю оценку текущего студента

        marks = student["marks"]

        avg_mark = sum(marks) / len(marks)


        # сравниваем с заданным значением

        if avg_mark >= min_avg_mark:

            filtered_students.append(student)


    return filtered_students


print("Все студенты:")

print_students(groupmates)


print(f"Студенты со средним баллом >= 4.0:")

high_achievers = filter_students_by_avg_mark(groupmates, 4.0)

print_students(high_achievers)
```

```
print(f"Студенты со средним баллом >= 3.5:")

good_students = filter_students_by_avg_mark(groupmates, 3.5)

print_students(good_students)
```

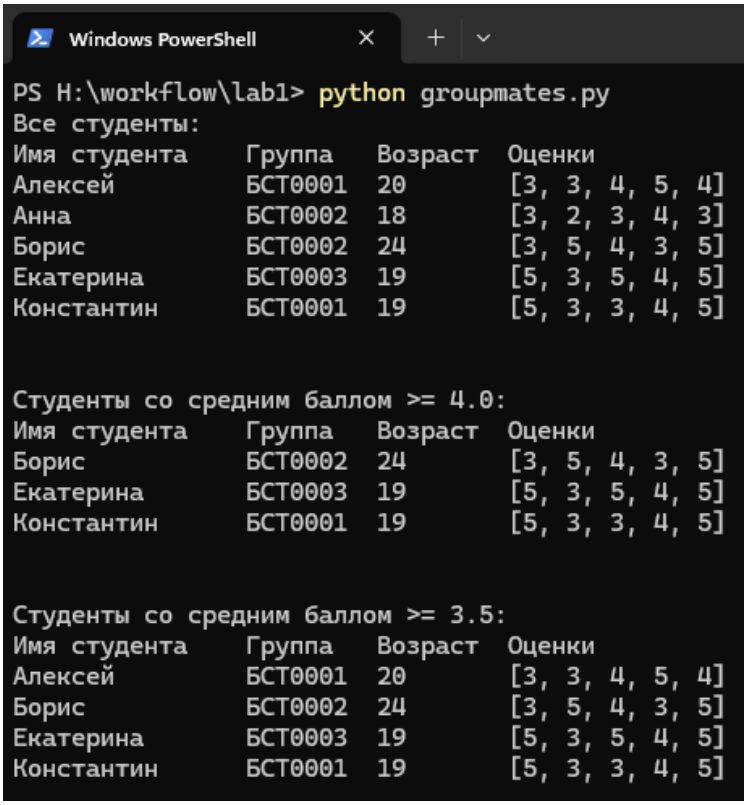

Рисунок 5 – Пример работы функции фильтрации студентов по средней оценке

2. Установка и запуск веб-фреймворка django

Установим и запустим веб-фреймворка django с помощью команды

```
python -m django startproject admin_learning
```

```
H:.
│   groupmates.py
│   helloworld.py
│
└───admin_learning
    │   manage.py
    │
    └───admin_learning
            asgi.py
            settings.py
            urls.py
            wsgi.py
            __init__.py
```

Рисунок 6 – Дерево нового проекта

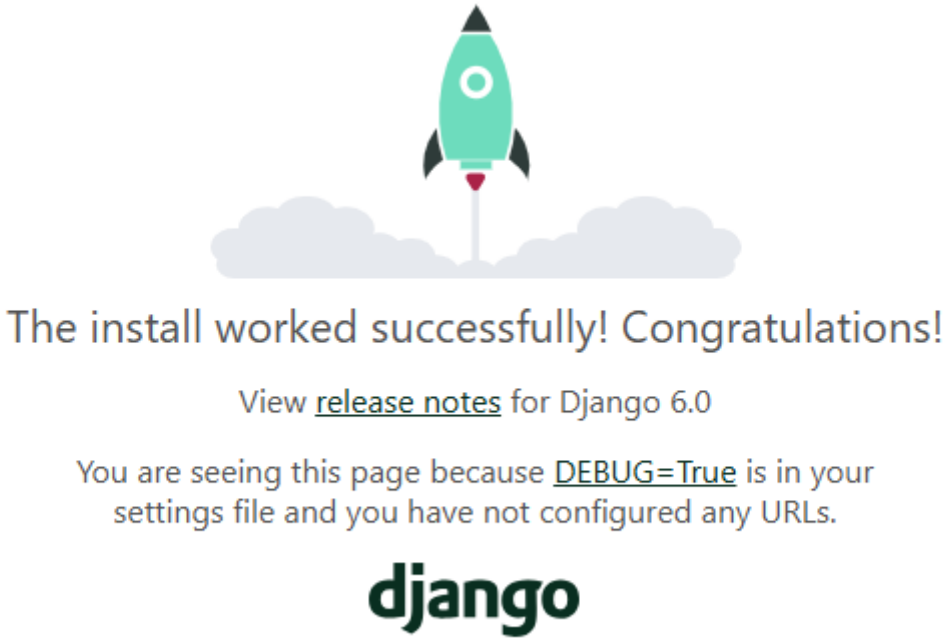

Рисунок 7 – Стартовая страница Django по адресу http://127.0.0.1:8000/

Теперь изучим файл settings.py через редактор IDLE

settings.py - H:\workflow\lab1\admin_learning\admin_learning\settings.py (3.13.5)

File Edit Format Run Options Window Help

```python
"""
Django settings for admin_learning project.

Generated by 'django-admin startproject' using Django 6.0.3.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-ehtljl5pqcld$qetalbtp6%(hcool5)y-yqd9dl&3==85=2w(l

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

Ln: 1 Col: 0

Рисунок 8 – Содержимое файла settings.py

Разберем состав данного файла:

BASE_DIR - является базовой директорией проекта

SECRET_KEY – ключ для криптографических операций, используемый для шифрования сессий, паролей, CSRF-токенов

DEBUG - Режим отладки

ALLOWED_HOSTS - Список разрешенных доменов для доступа к сайту

INSTALLED_APPS – функциональные приложения

MIDDLEWARE – промежуточный слой между запросом и ответом (легкая, низкоуровневая система плагинов для глобального изменения входных или выходных данных)

ROOT_URLCONF - файл с URL-маршрутами проекта

TEMPLATES – шаблоны, отвечающие за внешний вид данных в браузере конечного пользователя

DATABASES - БД для разработки

AUTH_PASSWORD_VALIDATORS – валидация паролей

STATIC_URL - URL для доступа к статическим файлам

В файле settings.py настроим два параметра для базы данных параметрам ENGINE и NAME.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db_admin_learning',
    }
}
```

Рисунок 9 – Указываем используемую БД и ее имя

Теперь в качестве БД будет использована sqlite3, а ее название db_admin_learning.

Исследование административного интерфейса django.

Войдем в административный интерфейс django, но для начала создадим все нужные таблицы в базе данных с помощью команды

```
python manage.py migrate
```

```
Windows PowerShell
PS H:\workflow\lab1\admin_learning> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
PS H:\workflow\lab1\admin_learning>
```

Рисунок 10 – Создание всех нужных таблиц в базе данных

Создадим суперпользователя

```
python manage.py createsuperuser
```

```
PS H:\workflow\lab1\admin_learning> python manage.py createsuperuser
Username (leave blank to use 'lebma'): Lebedev2251
Email address: lebmaxim@mail.ru
Password:
Password (again):
Superuser created successfully.
PS H:\workflow\lab1\admin_learning>
```

Рисунок 11 – Создание суперпользователя

Запустим сервер и перейдем по адресу http://127.0.0.1:8000/admin/

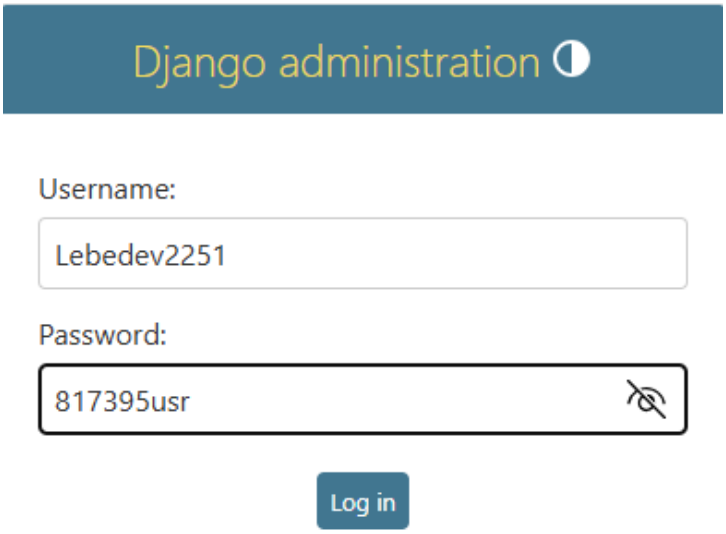

Рисунок 12 – Окно входа

После успешной авторизации открывается главная страница административной панели.

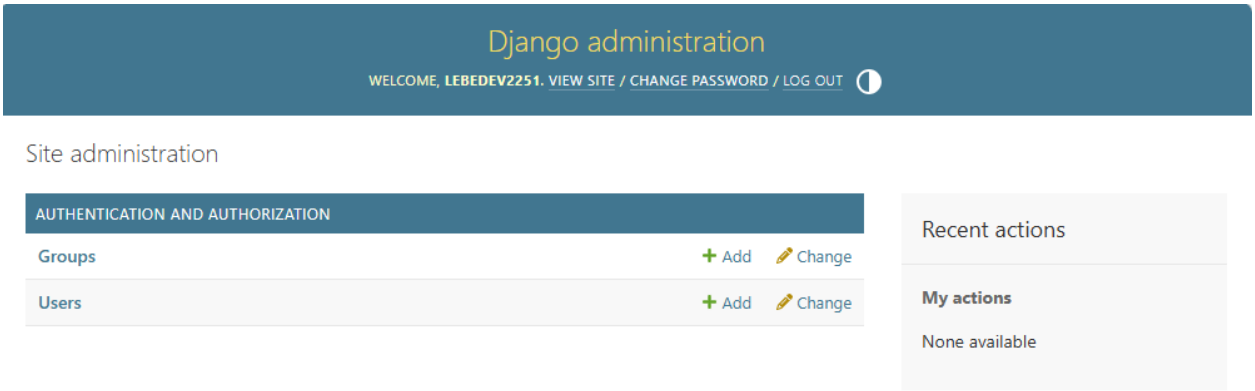

Рисунок 13 – Главная страница административной панели

По ссылке Users можно увидеть список всех зарегистрированных пользователей.

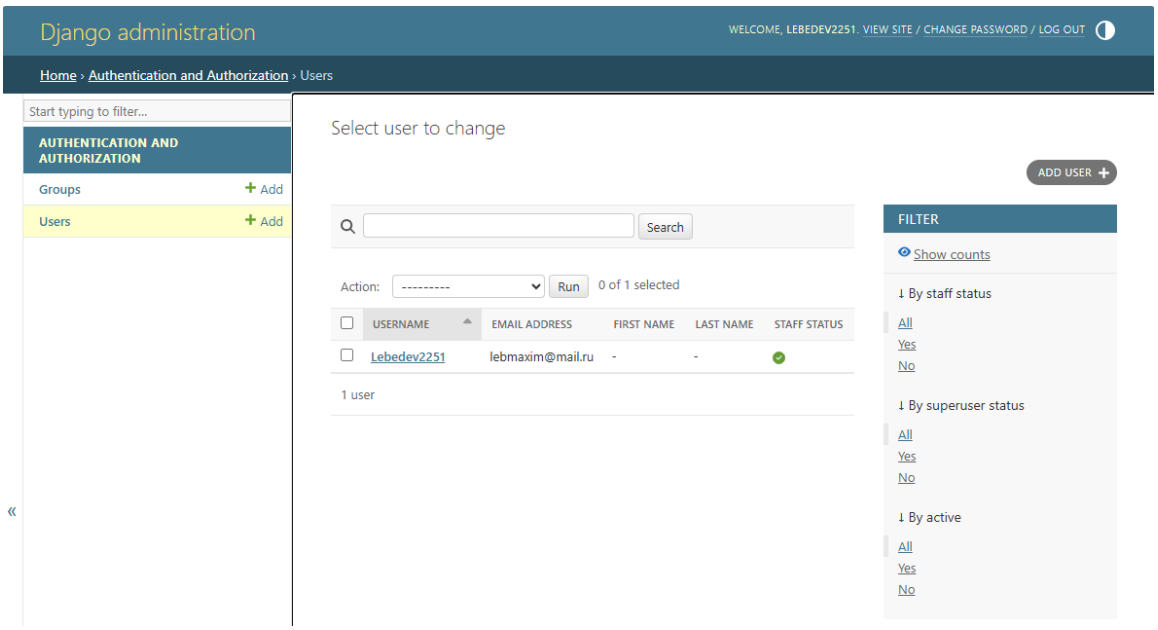

Рисунок 14 – Страница со списком всех зарегистрированных пользователей

Добавим нового пользователя

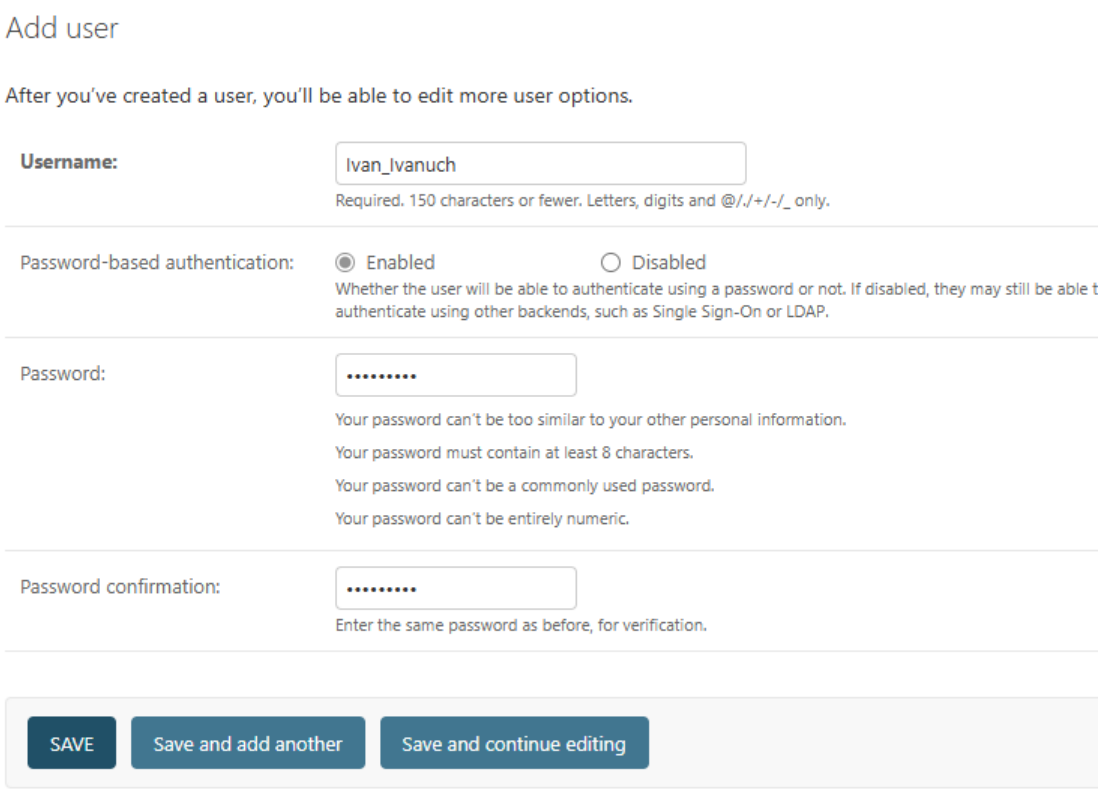

Рисунок 15 – Добавление нового пользователя

Затем наделим его правами суперюзера

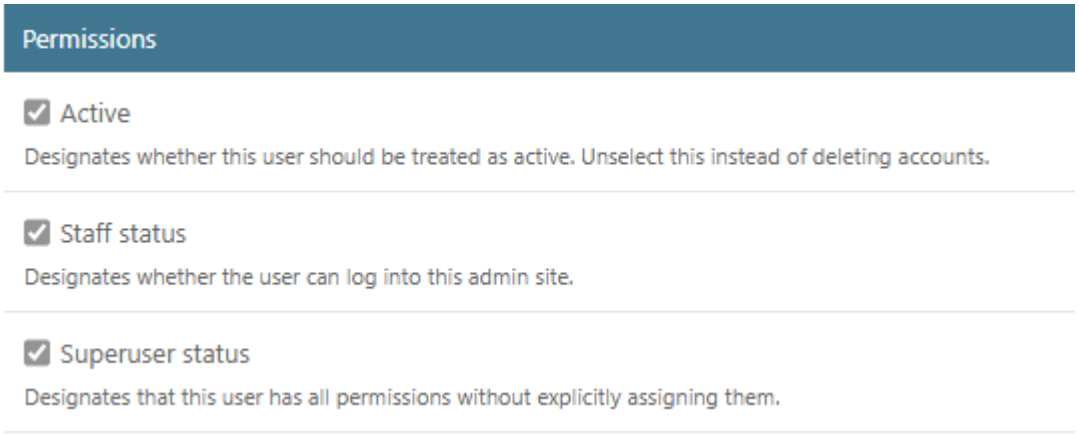

Рисунок 16 – Наделение правами суперюзера нового пользователя

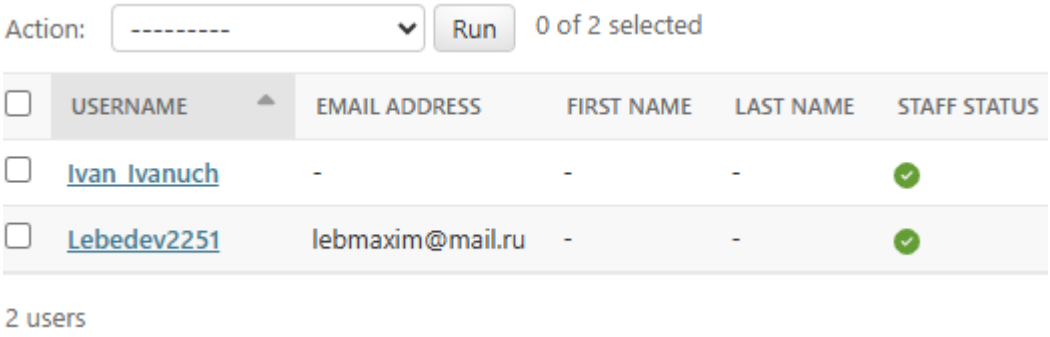

Рисунок 17 – Обновленный список всех пользователей

Далее добавим еще одного пользователя уже без прав суперюзера по такой же схеме, а после сделаем его неактивным путем снятия «галочки» напротив Active

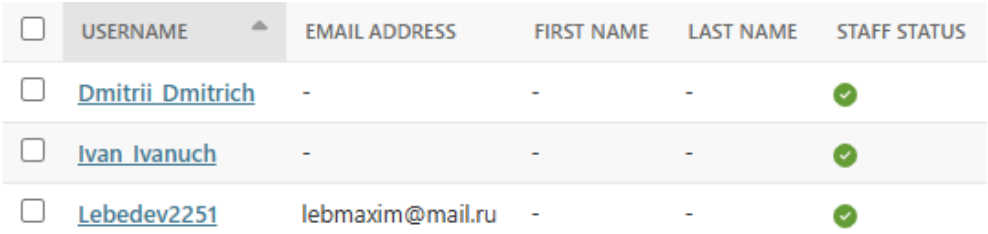

Рисунок 18 – Добавление еще одного пользователя

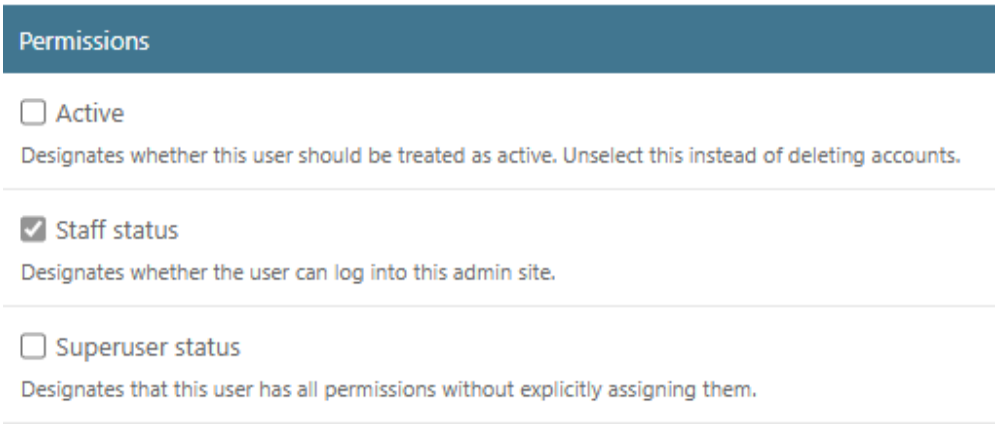

Рисунок 19 – Блокировка пользователя без удаления учетной записи

Лабораторная работа №2

1. Создание первой web-страницы с простым текстом.

Создадим в папке lab2 новый проект под названием helloworld с помощью команды:

```
python -m django startproject helloworld
```

затем выполним команду :

```
python manage.py startapp flatpages
```

```
PS H:\workflow\lab2> python -m django startproject helloworld
PS H:\workflow\lab2> cd helloworld
PS H:\workflow\lab2\helloworld> python manage.py startapp flatpages
PS H:\workflow\lab2\helloworld> cd..
PS H:\workflow\lab2> tree /f
Folder PATH listing for volume Elements
Volume serial number is 945C-ED20
H:.
└───helloworld
    │   manage.py
    │
    ├───flatpages
    │   │   admin.py
    │   │   apps.py
    │   │   models.py
    │   │   tests.py
    │   │   views.py
    │   │   __init__.py
    │   │
    │   └───migrations
    │           __init__.py
    │
    └───helloworld
        │   asgi.py
        │   settings.py
        │   urls.py
        │   wsgi.py
        │   __init__.py
        │
        └───__pycache__
                settings.cpython-313.pyc
                __init__.cpython-313.pyc
```

Рисунок 1 – Структура нового проекта

В качестве настроек БД в файле settings.py укажем:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db_helloworld',
    }
}
```

Добавим приложение flatpages в кортеж INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'flatpages',
]
```

А для будущей страницы в файле urls.py создадим новы адрес:

```
from django.contrib import admin
from django.urls import path, include
from flatpages import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.home, name='home'),
]
```

Настроим маршрут для главной страницы, назначив ему обработчик home из модуля flatpages.views. Затем создадим саму функцию-представление home в файле flatpages/views.py, которая будет формировать ответ для пользователя:

```
from django.http import HttpResponse


def home(request):
    return HttpResponse(u'Привет, Мир!')
```

Проверим результат по адресу http://127.0.0.1:8000/

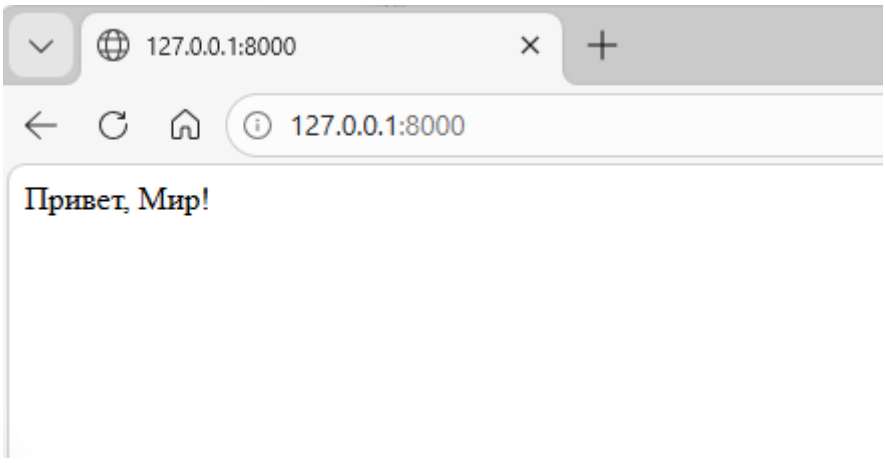

Рисунок 2 – Полученный ответ от первой страницы

Сделаем тоже и для адреса http://127.0.0.1:8000/hello/ добавив в urls.py новый маршрут:

```
path('hello/', views.hello, name='hello')
```

и в views.py такую же функцию, что и ранее для home:

```
def hello(request):
    return HttpResponse(u'Привет, Мир!')
```

Проверим результат:

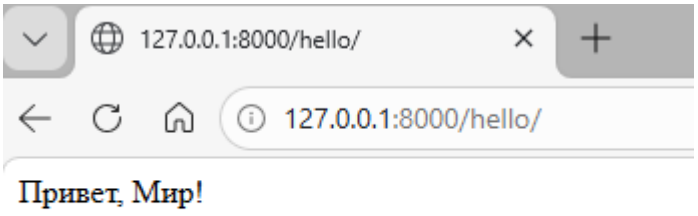

Рисунок 3 – Полученный ответ по адресу http://127.0.0.1:8000/hello/

2. Создание первого html-шаблона.

Создадим папку templates внутри приложения flatpages, внутри которого создадим файл index.html, добавив в него код:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Привет, Мир!</title>
    </head>
    <body>
```

```
        <h1>Привет, Мир!</h1>
        <h2>Это учебный сайт, с его помощью будут изучены технологии
python/django, html/css.</h2>
        <h3>Как видите, здесь используются заголовки различных
уровней.</h3>
        <p>Здесь есть маркированный список:</p>
        <ul>
            <li>Элемент 1;</li>
            <li>элемент 2;</li>
            <li>элемент 3;</li>
            <li>последний элемент.</li>
        </ul>
        <p>И нумерованный список:</p>
        <ol>
            <li>Элемент 1;</li>
            <li>элемент 2;</li>
            <li>элемент 3;</li>
            <li>последний элемент.</li>
        </ol>
        <p>И даже таблица:</p>
        <table border="1">
            <thead>
                <tr>
                    <th>Столбик 1</th>
                    <th>Столбик 2</th>
                    <th>Столбик 3</th>
                </tr>
            </thead>
            <tr>
                <td>Строка 1 Столбец 1</td>
                <td>Строка 1 Столбец 2</td>
                <td>Строка 1 Столбец 3</td>
            </tr>
            <tr>
                <td>Строка 2 Столбец 1</td>
                <td>Строка 2 Столбец 2</td>
                <td>Строка 2 Столбец 3</td>
```

```
            </tr>
            <tr>
                <td>Строка 3 Столбец 1</td>
                <td>Строка 3 Столбец 2</td>
                <td>Строка 3 Столбец 3</td>
            </tr>
        </table>
    </body>
</html>
```

Затем созданный html-файл подключим к функции представлению home, для этого в файле flatpages/views.py добавим операцию импортирования:

views.py - H:\workflow\lab2\helloworld\flatpages\views.py (3.13.5)

File Edit Format Run Options Window Help

```
1 from django.shortcuts import render
2 from django.http import HttpResponse
3
4 def home(request):
5     return render(request, 'index.html', {})
6 def hello(request):
7     return HttpResponse(u'Привет, Мир!')
8
```

Рисунок 4 – Подключение html-файла к функции home

Перезагрузим страницу:

127.0.0.1:8000

Привет, Мир!

Это учебный сайт, с его помощью будут изучены технологии python/django, html/css.

Как видите, здесь используются заголовки различных уровней.

Здесь есть маркированный список:

- Элемент 1;
- элемент 2;
- элемент 3;
- последний элемент.

И нумерованный список:

1. Элемент 1;
2. элемент 2;
3. элемент 3;
4. последний элемент.

И даже таблица:

Столбик 1	Столбик 2	Столбик 3
Строка 1 Столбец 1	Строка 1 Столбец 2	Строка 1 Столбец 3
Строка 2 Столбец 1	Строка 2 Столбец 2	Строка 2 Столбец 3
Строка 3 Столбец 1	Строка 3 Столбец 2	Строка 3 Столбец 3

Рисунок 5 – Полученная страница

Добавим к таблице две строки и один столбец, уберем все границы и сделаем заголовки списков подзаголовками четвертого уровня:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Привет, Мир!</title>
    </head>
    <body>
        <h1>Привет, Мир!</h1>
        <h2>Это учебный сайт, с его помощью будут изучены технологии
python/django, html/css.</h2>
        <h3>Как видите, здесь используются заголовки различных
уровней.</h3>

        <h4>Маркированный список:</h4>
        <ul>
```

```
    <li>Элемент 1;</li>
    <li>элемент 2;</li>
    <li>элемент 3;</li>
    <li>последний элемент.</li>
</ul>

<h4>Нумерованный список:</h4>
<ol>
    <li>Элемент 1;</li>
    <li>элемент 2;</li>
    <li>элемент 3;</li>
    <li>последний элемент.</li>
</ol>

<p>И даже таблица:</p>
<table border="0">
    <thead>
        <tr>
            <th>Столбик 1</th>
            <th>Столбик 2</th>
            <th>Столбик 3</th>
            <th>Столбик 4</th>
        </tr>
    </thead>
    <tr>
        <td>Строка 1 Столбец 1</td>
        <td>Строка 1 Столбец 2</td>
        <td>Строка 1 Столбец 3</td>
        <td>Строка 1 Столбец 4</td>
    </tr>
    <tr>
        <td>Строка 2 Столбец 1</td>
        <td>Строка 2 Столбец 2</td>
        <td>Строка 2 Столбец 3</td>
        <td>Строка 2 Столбец 4</td>
    </tr>
    <tr>
```

```
                <td>Строка 3 Столбец 1</td>
                <td>Строка 3 Столбец 2</td>
                <td>Строка 3 Столбец 3</td>
                <td>Строка 3 Столбец 4</td>
            </tr>
            <tr>
                <td>Строка 4 Столбец 1</td>
                <td>Строка 4 Столбец 2</td>
                <td>Строка 4 Столбец 3</td>
                <td>Строка 4 Столбец 4</td>
            </tr>
            <tr>
                <td>Строка 5 Столбец 1</td>
                <td>Строка 5 Столбец 2</td>
                <td>Строка 5 Столбец 3</td>
                <td>Строка 5 Столбец 4</td>
            </tr>
        </table>
    </body>
</html>
```

Проверим результат:

127.0.0.1:8000

Привет, Мир!

Это учебный сайт, с его помощью будут изучены технологии python/django, html/css.

Как видите, здесь используются заголовки различных уровней.

Маркированный список:

- Элемент 1;
- элемент 2;
- элемент 3;
- последний элемент.

Нумерованный список:

1. Элемент 1;
2. элемент 2;
3. элемент 3;
4. последний элемент.

И даже таблица:

Столбик 1	Столбик 2	Столбик 3	Столбик 4
Строка 1 Столбец 1	Строка 1 Столбец 2	Строка 1 Столбец 3	Строка 1 Столбец 4
Строка 2 Столбец 1	Строка 2 Столбец 2	Строка 2 Столбец 3	Строка 2 Столбец 4
Строка 3 Столбец 1	Строка 3 Столбец 2	Строка 3 Столбец 3	Строка 3 Столбец 4
Строка 4 Столбец 1	Строка 4 Столбец 2	Строка 4 Столбец 3	Строка 4 Столбец 4
Строка 5 Столбец 1	Строка 5 Столбец 2	Строка 5 Столбец 3	Строка 5 Столбец 4

Рисунок 6 – Обновленная страница

Создадим новый html-шаблон с названием «static_handler.html» с таким же содержимым для дальнейшей работы.

3. Настройка обработки статичных файлов для django.

Создадим папку static с директорией css, а внутрь этой директории добавим пустой файл index.css. В static_handler.html добавим строку:

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/index.css' %}">
```

для отображения картинки после (тега </h3>):

```
        <img src="{% static 'img/django-logo-positive.png' %}" alt="Django
logo" />
```

и в index.css

```
body {
    background: #1abc9c;
    font-family: Tahoma, Arial, sans-serif;
    color: #333;
}

table {
    border-collapse: collapse;
}

p, h4 {
    font-size: 22px;
    margin-bottom: 0;
}

ul, ol {
    margin: 0;
}

table tr td {
    padding: 5px;
}
```

а в urls.py изменим путь на

```
    path('', views.static_handler, name='home'),
```

Проверим релультат:

Рисунок 7 – Обновленная страница с css-стилями

Далее установим для заголовка первого уровня шрифт с засечками, сделаем картинку высотой 30px и ширину таблицы равно 100% экрана. Для этого внесем изменения в файл index.css:

```
body {
    background: #1abc9c;
    font-family: Tahoma, Arial, sans-serif;
    color: #333;
}

table {
    border-collapse: collapse;
```

```
        width: 100%;
}

p, h4 {
    font-size: 14px;
    margin-bottom: 0;
}

h1 {
    font-family: Georgia, 'Times New Roman', Times, serif;
}

ul, ol {
    margin: 0;
}

table tr td {
    padding: 5px;
}

img {
    height: 30px;
}
```

Проверим результат:

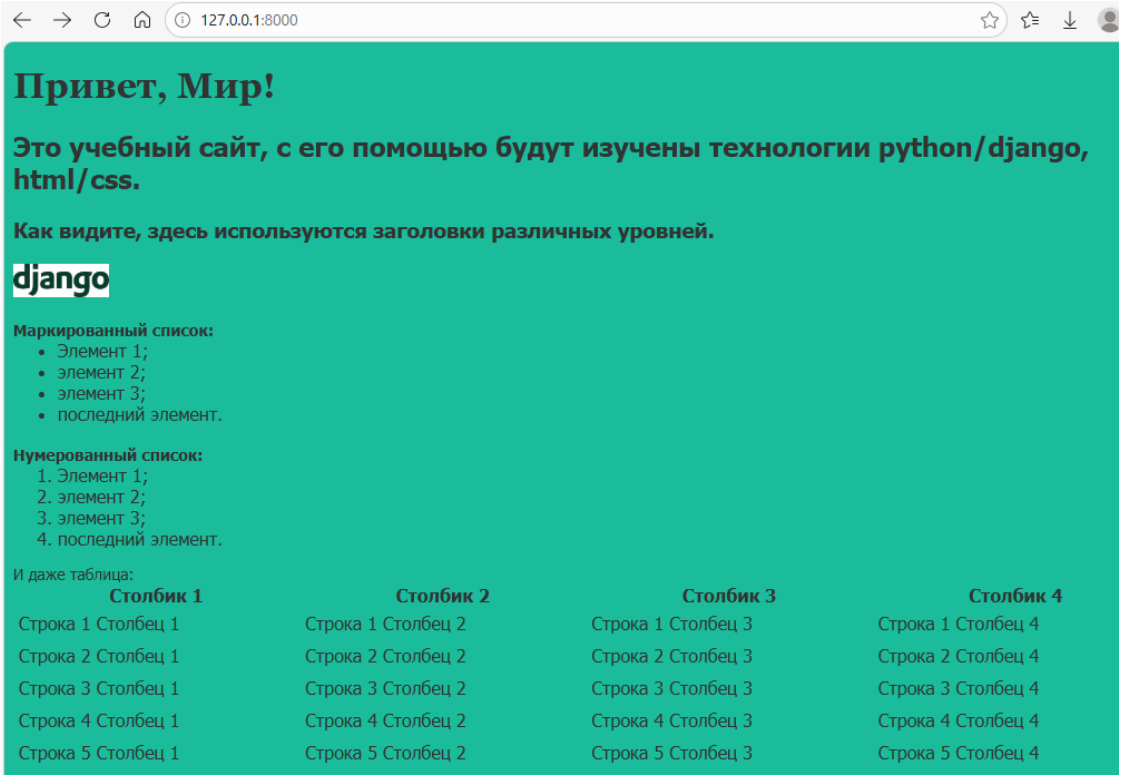

Рисунок 8 – Обновленная страница

Лабораторная работа №3.

1. Создание первой модели данных и её регистрация в административном приложении django.

Создадим новые проект в папке lab3 проект под названием blog по раннее уже описанному методу. Выполним команду

```
python manage.py startapp articles
```

и

```
python -m django startproject blog
```

для создания в папке blog нового приложения.

Выполним базовые настройки проекта как в предыдущих работах.

Откроем articles/models.py, в котором напишем:

```
from django.db import models
from django.contrib.auth.models import User

class Article(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    text = models.TextField()
    created_date = models.DateField(auto_now_add=True)

    def __str__(self):
        return "%s: %s" % (self.author.username, self.title)

    def get_excerpt(self):
        return self.text[:140] + "..." if len(self.text) > 140 else self.text
```

Затем в articles/admin.py добавим следующий функционал:

```
from django.contrib import admin
```

```
from .models import Article

class ArticleAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'get_excerpt', 'created_date')

admin.site.register(Article, ArticleAdmin)
```

Авторизируемся на http://127.0.0.1:8000/admin/ и перейдем на вкладку Articles и добавим статьи:

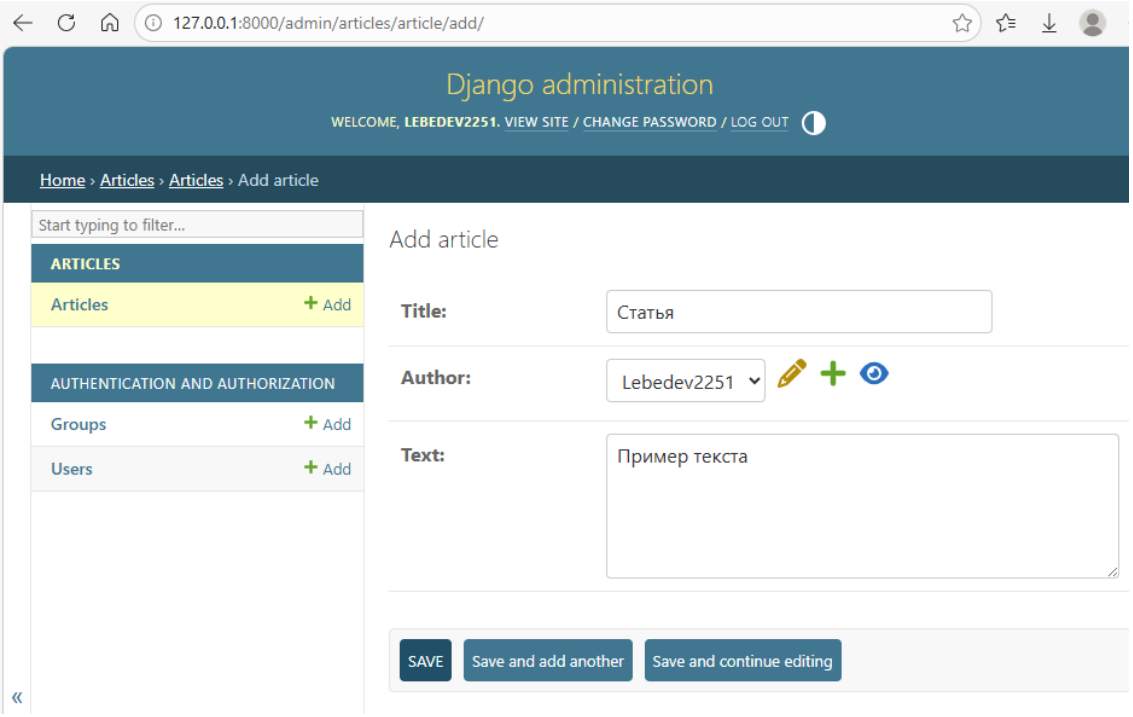

Рисунок 9 – Добавление новой статьи

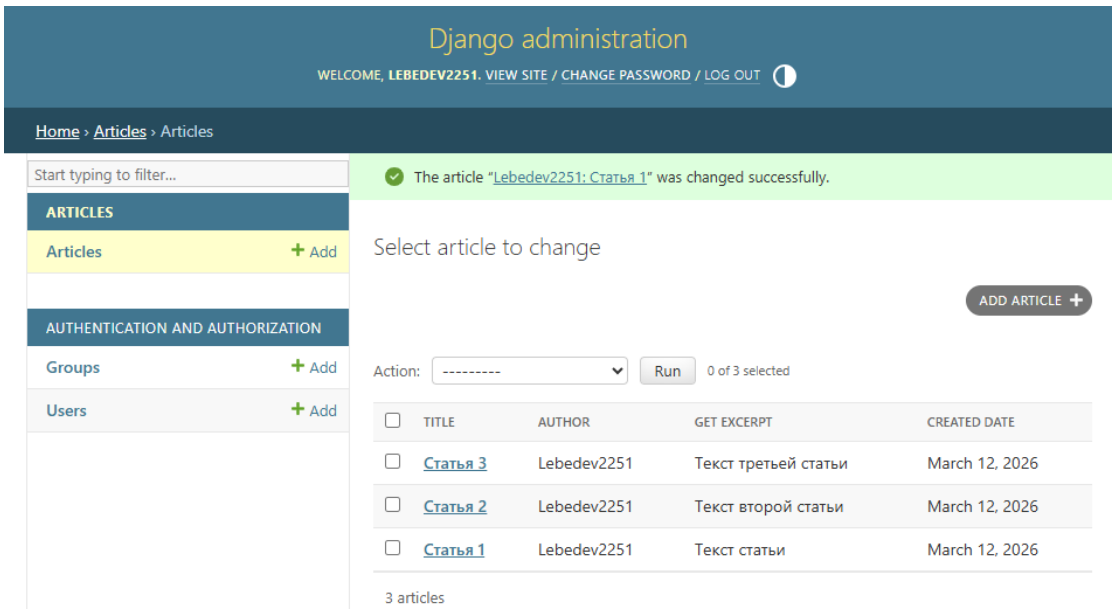

Рисунок 10 – Список всех статей

Откроем файл БД db_blog и изменим текст одной записи и название статьи для другой:

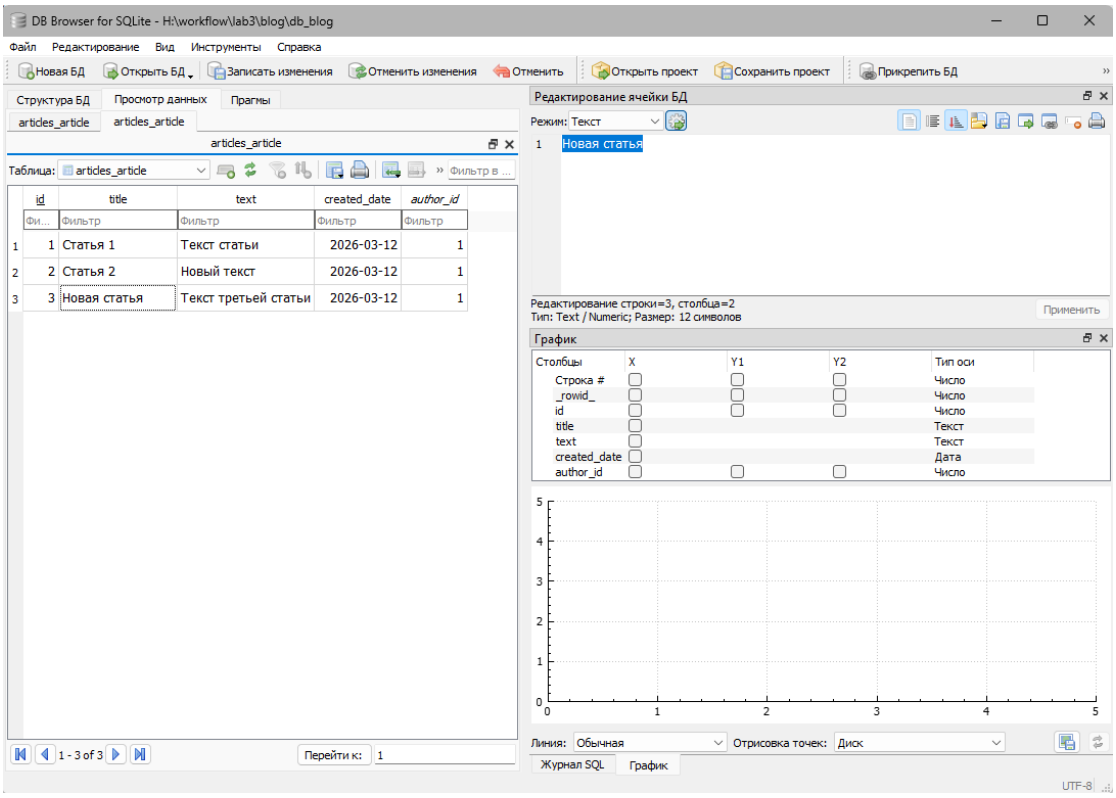

Рисунок 11 – Изменение в файле БД

Проверяем изменения:

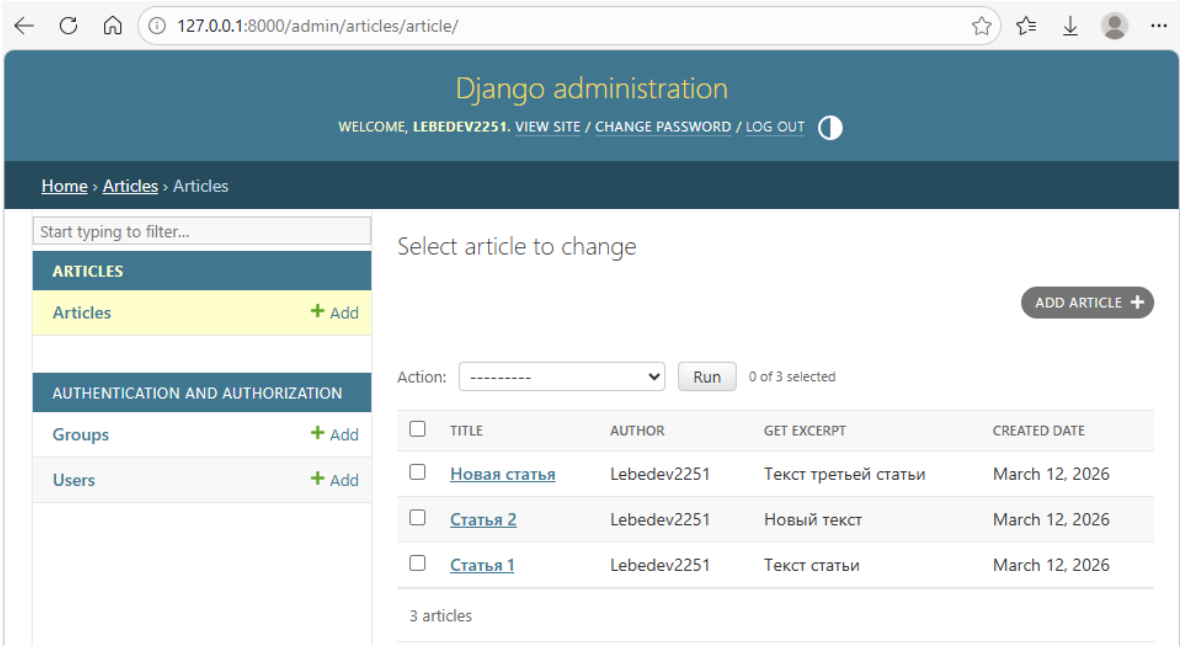

Рисунок 12 – Страница со списком статей

2. Динамическое генерирование шаблона для вывода всех экземпляров этой модели.

В директории articles создадим папку templates, внутри которой создадим файл archive.html.

```
<!DOCTYPE html>
<html>
<head>
    <title>Архив всех статей</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/index.css' %}">
</head>
<body>
    <div class="header">
        <img src="{% static 'img/django-logo-positive.png' %}" />
    </div>
    <div class="archive">
```

```
        {% for post in posts %}
        <div class="one-post">
            <h2 class="post-title">{{ post.title }}</h2>
            <div class="article-info">
                <div class="article-author">{{ post.author.username }}</div>
                <div class="article-created-date">{{ post.created_date
}}</div>
            </div>
            <p class="article-text">{{ post.get_excerpt }}</p>
        </div>
        {% endfor %}
    </div>
</body>
</html>
```

для urls.py:

```
from django.contrib import admin
from django.urls import path
from articles import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.archive, name='archive'),
]
```

Перейдем на главную страницу:

Рисунок 13 – Полученная страница со статьями

Откроем файл базы данных, где хранятся экземпляры статей к текущему проекту и добавим новую запись в блог непосредственно через менеджер базы данных:

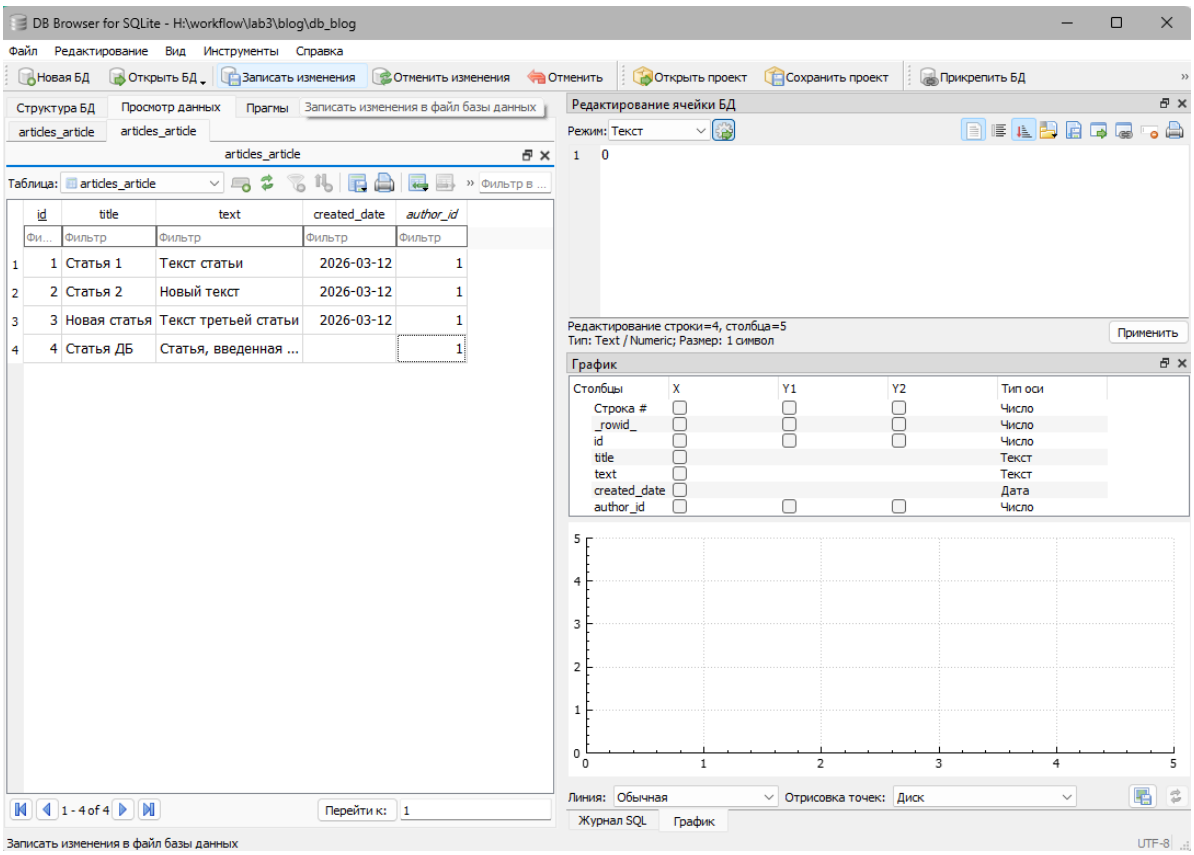

Рисунок 14 – Создание записи непосредственно в БД

Проверяем результат на главной странице:

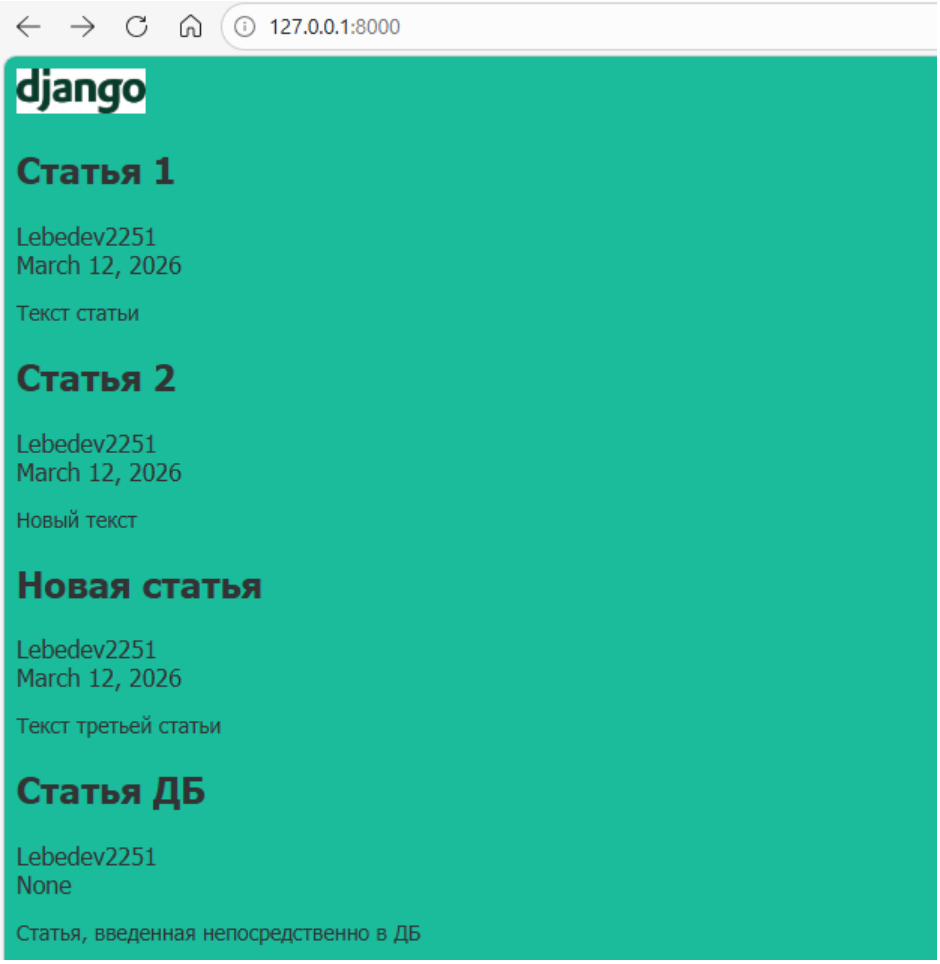

Рисунок 15 – Успешно добавленная статья

Лабораторная работа №4.

1. Создание страницы определенной записи.

Создадим в папке lab4 новый проект под названием blog и скопируем приложение articles из предыдущего проекта. В INSTALLED_APPS и добавим необходимые настройки адресов:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles',
]
```

, а в blog/urls.py укажем следующее регулярное выражение:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.archive, name='archive'),
    re_path(r'^article/(?P<article_id>\d+)$', views.get_article,
name='get_article'),
]
```

Затем создадим в файле articles/views.py ещё функцию get_article:

```
from django.shortcuts import render
from django.http import Http404
from .models import Article

def archive(request):
    return render(request, 'archive.html', {"posts": Article.objects.all()})

def get_article(request, article_id):
    try:
        post = Article.objects.get(id=article_id)
        return render(request, 'article.html', {"post": post})
    except Article.DoesNotExist:
        raise Http404
```

и для введем изменения в article.html, используя тег <a> в атрибуте href с указанием адреса перехода:

```
<!DOCTYPE html>
```

```
<html>
<head>
    <title>{{ post.title }}</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/article.css' %}">
</head>
<body class="article">
    <div class="header">
        <img src="{% static 'img/django-logo-positive.png' %}" />
    </div>
    <div class="article-full">
        <h1>{{ post.title }}</h1>
        <div class="article-info">
            <div class="article-author">{{ post.author.username }}</div>
            <div class="article-created-date">{{ post.created_date }}</div>
        </div>
        <div class="article-text">
            {{ post.text }}
        </div>
        <p><a href="{% url 'archive' %}">Back</a></p>
    </div>
</body>
</html>
```

и создадим archive.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Archive</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/article.css' %}">
</head>
<body class="archive">
    <div class="header">
        <img src="{% static 'img/django-logo-positive.png' %}" />
    </div>
    <div class="archive">
        {% for post in posts %}
        <div class="one-post">
            <h2 class="post-title">
                <a href="{% url 'get_article' article_id=post.id %}">{{
post.title }}</a>
            </h2>
            <div class="article-info">
                <div class="article-author">{{ post.author.username }}</div>
                <div class="article-created-date">{{ post.created_date
}}</div>
```

```
            </div>
            <p class="article-text">{{ post.get_excerpt }}</p>
        </div>
        {% endfor %}
    </div>
</body>
</html>
```

2. Верстка обеих страниц в соответствии с макетом

Придадим внешние стили страницам, создав article.css в директории static/css/:

```
body {
    background: #1abc9c;
    font-family: Tahoma, Arial, sans-serif;
    color: #ffffff;
}

.header img {
    display: block;
    width: 318px;
    margin-left: auto;
    margin-right: auto;
}

.archive {
    width: 960px;
    margin-left: auto;
    margin-right: auto;
}

.post-title a {
    color: #ffffff;
}

.article-author {
    width: 50%;
    float: left;
}

.article-created-date {
    text-align: right;
}
```

```
.article-full {
    width: 960px;
    margin-left: auto;
    margin-right: auto;
}

.article-text {
    clear: both;
    padding-top: 20px;
}
```

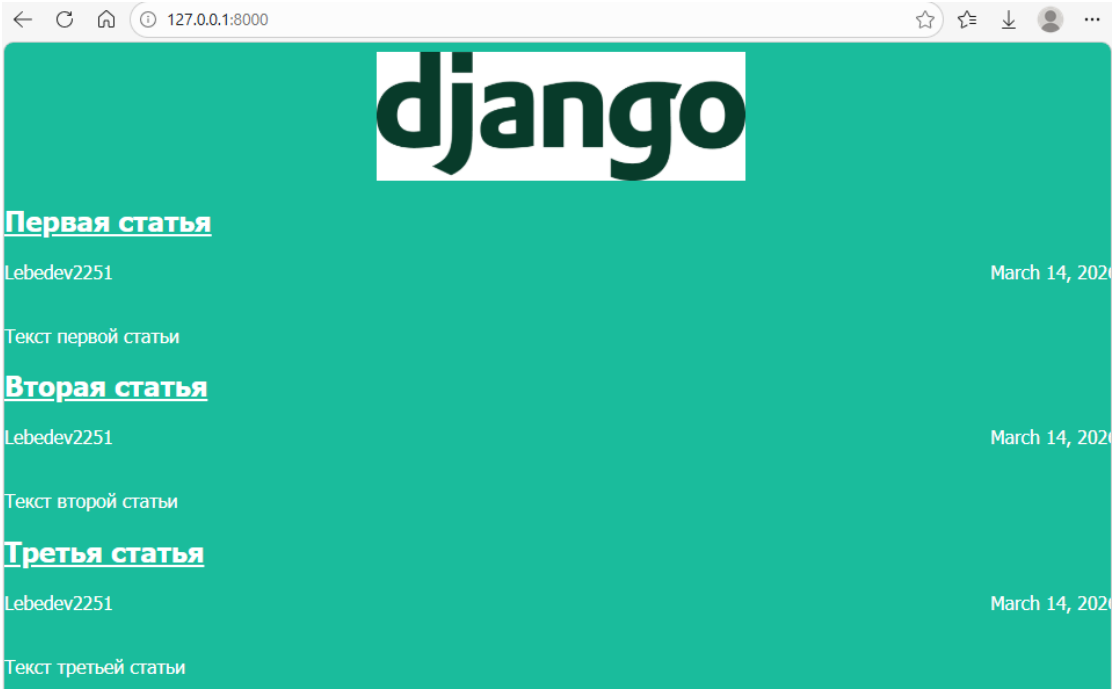

Рисунок 1 – Главная страница со всеми статьями

Рисунок 2 – Выбранная статья

Создадим стили, которые соответствуют макету, для страницы определенной записи, добавив для этого соответствующую ссылку в article.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>{{ post.title }}</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/article.css' %}">
</head>
<body class="article">
    <div class="header">
        <img src="{% static 'img/django-logo-positive.png' %}" />
    </div>
    <div class="article-full">
        <h1>{{ post.title }}</h1>
        <div class="article-info">
            <div class="article-author">{{ post.author.username }}</div>
            <div class="article-created-date">{{ post.created_date }}</div>
        </div>
        <div class="article-text">
            {{ post.text }}
        </div>
        <p class="back-link"><a href="{% url 'archive' %}">All posts</a></p>
    </div>
</body>
</html>
```

и стили для ссылки в файле article.css:

```
.back-link {
    margin-top: 30px;
    text-align: center;
}

.back-link a {
    color: #ffffff;
    text-decoration: none;
    font-weight: bold;
    padding: 10px 20px;
    border: 2px solid #ffffff;
    border-radius: 5px;
}
```

```
.back-link a:hover {
    background-color: #ffffff;
    color: #1abc9c;
}
```

Лабораторная работа №5

1. Создание формы и представления для нового поста.

Добавим в urls.py ещё один адрес: «article/new/», который будет обрабатываться функцией-представлением create_post:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.archive, name='archive'),
    re_path(r'^article/(?P<article_id>\d+)/$', views.get_article,
name='get_article'),
    path('article/new/', views.create_post, name='create_post'),
]
```

Затем создадим функцию create_post в articles/views.py, которая при первом обращении к странице будет возвращать html-код с формой создания новой записи, а при повторном обращении, если пользователь при этом заполнил форму с названием и текстом нового поста, представление должно сохранять отправленные через форму данные в БД:

```
def create_post(request):
    if request.method == "POST":
        article = Article(
            title=request.POST['title'],
            text=request.POST['text'],
            author=request.user
        )
        article.save()
        return HttpResponseRedirect(reverse('get_article',
args=(article.id,)))
    else:
        return render(request, 'create_post.html', {})
```

Создадим новый шаблон с формой «create_post.html»:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Django Public Blog - create article</title>
        {% load static %}
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body>
        <div class="header">
```

```
            <img src="{% static 'img/django-logo-positive.png' %}" />
        </div>
        <div class="content">
            <h1>Create new article</h1>
            <form method="POST">
                {% csrf_token %}
                <div>
                    <input type="text" name="title" placeholder="Article
title" value="{{ form.title }}">
                </div>
                <div>
                    <textarea name="text" placeholder="Article text">{{
form.text }}</textarea>
                </div>
                <div>
                    <input type="submit" value="Save">
                </div>
            </form>
            {% if form.errors %}
                <div class="error">{{ form.errors }}</div>
            {% endif %}
            <p><a href="{% url 'archive' %}">Back to archive</a></p>
        </div>
    </body>
</html>
```

Добавим стили для шаблона в article.css:

```
.content {
    width: 960px;
    margin-left: auto;
    margin-right: auto;
}

input[type="text"], textarea {
    width: 100%;
    padding: 8px;
    margin-bottom: 15px;
    border: 1px solid #ddd;
    border-radius: 4px;
    box-sizing: border-box;
}

textarea {
    min-height: 200px;
```

```
}

input[type="submit"] {
    background-color: #ffffff;
    color: #1abc9c;
    border: 2px solid #ffffff;
    border-radius: 5px;
    padding: 10px 20px;
    font-weight: bold;
    cursor: pointer;
}

input[type="submit"]:hover {
    background-color: transparent;
    color: #ffffff;
}

.error {
    color: #ffcccc;
    margin-top: 15px;
    padding: 10px;
    border: 1px solid #ffcccc;
    border-radius: 4px;
}
```

Рисунок 1 – Форма для создания новой статьи

Создадим стили, подключив css-файл к шаблону, а также добавим проверку, что введенное для нового поста название уникально. Вначале обновим шаблон create_post.html:

```
<!DOCTYPE html>
```

```
<html>
    <head>
        <title>Django Public Blog - create article</title>
        {% load static %}
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body>
        <div class="header">
            <img src="{% static 'img/django-logo-positive.png' %}" />
        </div>
        <div class="content">
            <h1>Create new article</h1>
            <form method="POST">
                {% csrf_token %}
                <div class="form-group">
                    <label for="title">Title:</label>
                    <input type="text" name="title" id="title"
placeholder="Enter article title" value="{{ form.title }}">
                </div>
                <div class="form-group">
                    <label for="text">Text:</label>
                    <textarea name="text" id="text" placeholder="Enter
article text">{{ form.text }}</textarea>
                </div>
                <div class="form-group">
                    <input type="submit" value="Save">
                </div>
            </form>
            {% if form.errors %}
                <div class="error">{{ form.errors }}</div>
            {% endif %}
            <p><a href="{% url 'archive' %}">Back to archive</a></p>
        </div>
    </body>
</html>
```

затем обновим функцию create_post в views.py для проверки уникальности названия нового поста:

```
def create_post(request):
    if not request.user.is_authenticated:
        raise Http404

    if request.method == "POST":
        form = {
            'text': request.POST["text"],
```

```
            'title': request.POST["title"]
        }

        if form["text"] and form["title"]:
            if Article.objects.filter(title=form["title"]).exists():
                form['errors'] = "Article with this title already exists"
                return render(request, 'create_post.html', {'form': form})

            article = Article.objects.create(
                text=form["text"],
                title=form["title"],
                author=request.user
            )
            return redirect('get_article', article_id=article.id)
        else:
            form['errors'] = "Not all fields are filled"
            return render(request, 'create_post.html', {'form': form})
    else:
        return render(request, 'create_post.html', {})
```

добавим стили для для формы создания статьи article.css:

```
.content {
    width: 960px;
    margin-left: auto;
    margin-right: auto;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    font-weight: bold;
    margin-bottom: 8px;
    font-size: 16px;
}

.form-group input[type="text"],
.form-group textarea {
    width: 100%;
    padding: 12px;
    border: 2px solid #ffffff;
    border-radius: 8px;
    background-color: rgba(255, 255, 255, 0.1);
```

```
    color: #ffffff;
    font-size: 16px;
    box-sizing: border-box;
    transition: all 0.3s ease;
}

.form-group input[type="text"]:focus,
.form-group textarea:focus {
    outline: none;
    background-color: rgba(255, 255, 255, 0.2);
    box-shadow: 0 0 10px rgba(255, 255, 255, 0.3);
}

.form-group input[type="text"]::placeholder,
.form-group textarea::placeholder {
    color: rgba(255, 255, 255, 0.7);
    font-style: italic;
}

.form-group textarea {
    min-height: 250px;
    resize: vertical;
}

.form-group input[type="submit"] {
    background-color: #ffffff;
    color: #1abc9c;
    border: 2px solid #ffffff;
    border-radius: 8px;
    padding: 12px 30px;
    font-weight: bold;
    font-size: 18px;
    cursor: pointer;
    transition: all 0.3s ease;
    width: auto;
    min-width: 200px;
}

.form-group input[type="submit"]:hover {
    background-color: transparent;
    color: #ffffff;
    transform: scale(1.05);
}

.error {
    color: #ffcccc;
    background-color: rgba(255, 0, 0, 0.1);
```

```
    margin-top: 20px;
    padding: 15px;
    border: 2px solid #ffcccc;
    border-radius: 8px;
    text-align: center;
    font-weight: bold;
}

h1 {
    text-align: center;
    margin-bottom: 40px;
    font-size: 36px;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.2);
}

p a {
    color: #ffffff;
    text-decoration: none;
    font-weight: bold;
    border-bottom: 2px solid transparent;
    transition: border-bottom 0.3s ease;
}

p a:hover {
    border-bottom: 2px solid #ffffff;
}
```

Проверим работу функции:

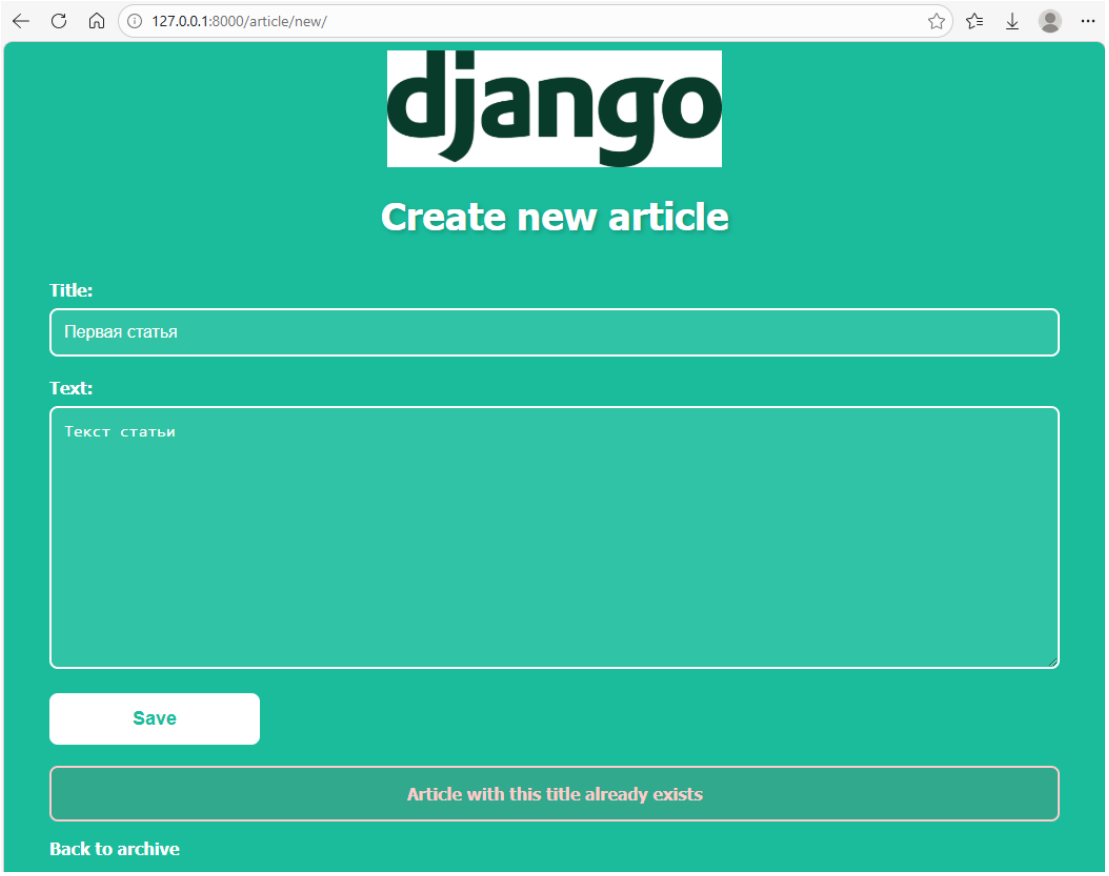

Рисунок 2 – Попытка создать статью с уже существующим заголовком

Лабораторная работа №6.

1. Создание формы, шаблона и представления для регистрации.

Для создания формы для страницы регистрации добавим новый маршрут в urls.py:

```
    path('register/', views.register, name='register'),
```

далее создадим функцию register в articles/views.py:

```
def register(request):
    if request.method == "POST":
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        password_confirm = request.POST.get('password_confirm')

        errors = {}

        if not username:
            errors['username'] = "Username is required"
        if not email:
            errors['email'] = "Email is required"
        if not password:
            errors['password'] = "Password is required"
        if not password_confirm:
            errors['password_confirm'] = "Password confirmation is required"

        if password and password_confirm and password != password_confirm:
            errors['password_match'] = "Passwords do not match"

        if username:
            try:
                User.objects.get(username=username)
                errors['username_exists'] = "Username already exists"
            except User.DoesNotExist:
                pass

        if errors:
            return render(request, 'register.html', {
                'errors': errors,
                'username': username,
                'email': email
            })

        user = User.objects.create_user(username, email, password)
        login(request, user)
```

```
        return redirect('archive')

    return render(request, 'register.html', {})
```

создадим шаблон register.html с проверкой полей:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Registration</title>
        {% load static %}
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body>
        <div class="header">
            <img src="{% static 'img/django-logo-positive.png' %}" />
        </div>
        <div class="content">
            <h1>Registration</h1>
            <form method="POST" class="register-form">
                {% csrf_token %}

                <div class="form-group">
                    <label for="username">Username:</label>
                    <input type="text" name="username" id="username"
                           placeholder="Enter username" value="{{ username
}}">
                    {% if errors.username %}
                        <div class="field-error">{{ errors.username }}</div>
                    {% endif %}
                    {% if errors.username_exists %}
                        <div class="field-error">{{ errors.username_exists
}}</div>
                    {% endif %}
                </div>

                <div class="form-group">
                    <label for="email">Email:</label>
                    <input type="email" name="email" id="email"
                           placeholder="Enter email" value="{{ email }}">
                    {% if errors.email %}
                        <div class="field-error">{{ errors.email }}</div>
                    {% endif %}
                </div>

                <div class="form-group">
                    <label for="password">Password:</label>
```

```
                    <input type="password" name="password" id="password"
                           placeholder="Enter password">
                    {% if errors.password %}
                        <div class="field-error">{{ errors.password }}</div>
                    {% endif %}
                </div>

                <div class="form-group">
                    <label for="password_confirm">Confirm password:</label>
                    <input type="password" name="password_confirm"
id="password_confirm"
                           placeholder="Confirm password">
                    {% if errors.password_confirm %}
                        <div class="field-error">{{ errors.password_confirm
}}</div>
                    {% endif %}
                    {% if errors.password_match %}
                        <div class="field-error">{{ errors.password_match
}}</div>
                    {% endif %}
                </div>

                <div class="form-group">
                    <input type="submit" value="Register">
                </div>
            </form>

            <p class="back-link"><a href="{% url 'archive' %}">Back to
archive</a></p>
        </div>
    </body>
</html>
```

ссылка после заголовка в archive.html и в article.html:

```
        <div class="header-links">
            <a href="{% url 'register' %}" class="register-link">Register</a>
        </div>
```

и стили в article.css для ссылки для регистрации, ее формы и полей:

```
.header {
    position: relative;
}

.header-links {
    position: absolute;
```

```
    top: 20px;
    right: 20px;
}

.register-link {
    color: #ffffff;
    text-decoration: none;
    font-weight: bold;
    padding: 8px 16px;
    border: 2px solid #ffffff;
    border-radius: 5px;
    transition: all 0.3s ease;
}

.register-link:hover {
    background-color: #ffffff;
    color: #1abc9c;
}

.register-form {
    max-width: 500px;
    margin: 0 auto;
}

.field-error {
    color: #ffcccc;
    font-size: 14px;
    margin-top: 5px;
    padding: 5px;
    background-color: rgba(255, 0, 0, 0.1);
    border-radius: 4px;
}

.form-group input[type="text"],
.form-group input[type="email"],
.form-group input[type="password"],
.form-group textarea {
    width: 100%;
    padding: 12px;
    border: 2px solid #ffffff;
    border-radius: 8px;
    background-color: rgba(255, 255, 255, 0.1);
    color: #ffffff;
    font-size: 16px;
    box-sizing: border-box;
    transition: all 0.3s ease;
}
```

```
.form-group input[type="text"]:focus,
.form-group input[type="email"]:focus,
.form-group input[type="password"]:focus,
.form-group textarea:focus {
    outline: none;
    background-color: rgba(255, 255, 255, 0.2);
    box-shadow: 0 0 10px rgba(255, 255, 255, 0.3);
}

.form-group input[type="text"]::placeholder,
.form-group input[type="email"]::placeholder,
.form-group input[type="password"]::placeholder,
.form-group textarea::placeholder {
    color: rgba(255, 255, 255, 0.7);
    font-style: italic;
}
```

Проверим работу формы:

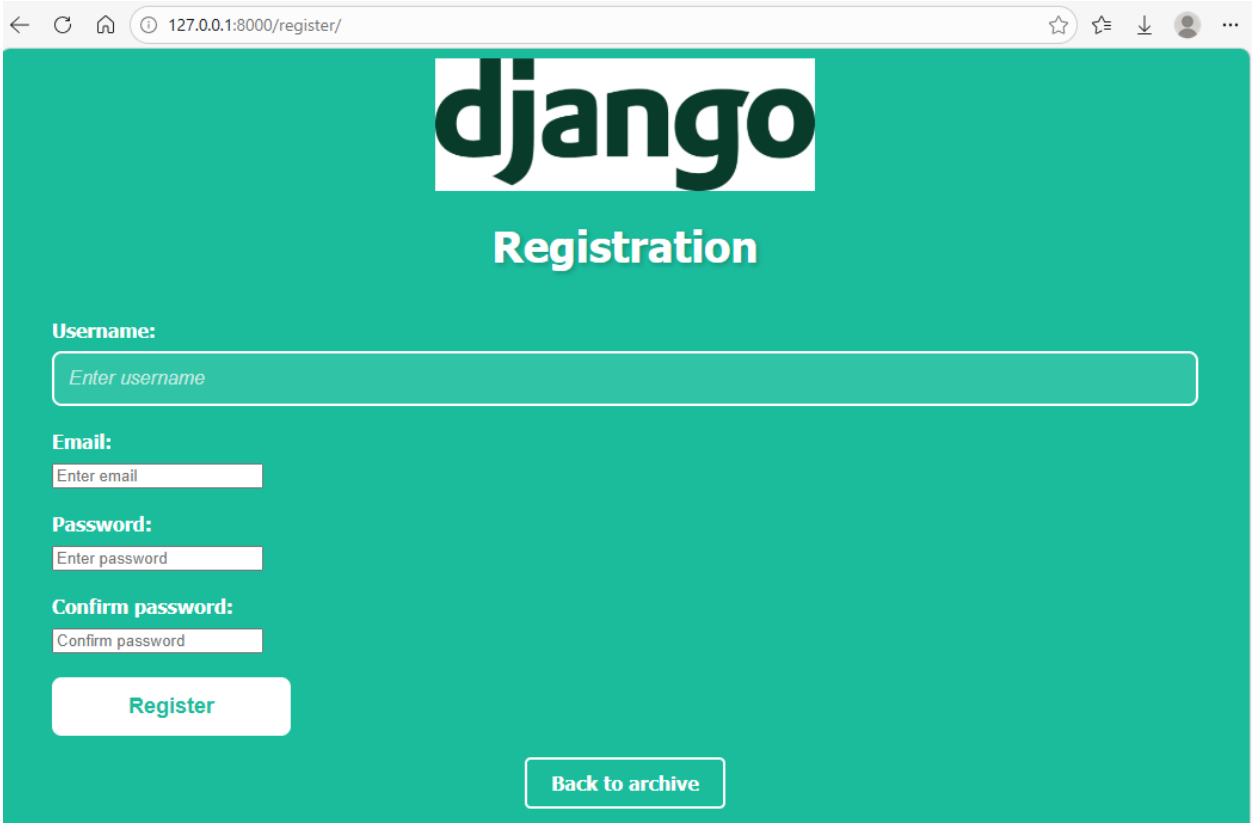

Рисунок 1 – Форма для регистрации нового пользователя

2. Создание формы, шаблона и представления для авторизации.

Далее необходимо создать шаблон и настроить адрес для отображения страницы авторизации. Для этого создадим представление, которое обрабатывает поступающие запросы и авторизует пользователей, проверяя заполнение полей и то, что логин и пароль соответствуют одному из аккаунтов проекта.

Добавляем новый путь в urls.py:

```
path('login/', views.user_login, name='login'),
```

Создадим функцию «user_login» в articles/views.py:

```
def register(request):
    if request.method == "POST":
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        password_confirm = request.POST.get('password_confirm')

        errors = {}

        if not username:
            errors['username'] = "Username is required"
        if not email:
            errors['email'] = "Email is required"
        if not password:
            errors['password'] = "Password is required"
        if not password_confirm:
            errors['password_confirm'] = "Password confirmation is required"

        if password and password_confirm and password != password_confirm:
            errors['password_match'] = "Passwords do not match"

        if username:
            try:
                User.objects.get(username=username)
                errors['username_exists'] = "Username already exists"
            except User.DoesNotExist:
                pass

        if errors:
            return render(request, 'register.html', {
                'errors': errors,
                'username': username,
                'email': email
```

```
            })

        user = User.objects.create_user(username, email, password)
        login(request, user)
        return redirect('archive')

    return render(request, 'register.html', {})

def user_login(request):
    if request.method == "POST":
        username = request.POST.get('username')
        password = request.POST.get('password')

        errors = {}

        if not username:
            errors['username'] = "Username is required"
        if not password:
            errors['password'] = "Password is required"

        if username and password:
            user = authenticate(request, username=username,
password=password)
            if user is not None:
                login(request, user)
                return redirect('archive')
            else:
                errors['invalid'] = "Invalid username or password"

        return render(request, 'login.html', {
            'errors': errors,
            'username': username
        })

    return render(request, 'login.html', {})
```

создадим шаблон для авторизации login.html:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Login</title>
        {% load static %}
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body>
```

```
        <div class="header">
            <img src="{% static 'img/django-logo-positive.png' %}" />
            <div class="header-links">
                <a href="{% url 'register' %}" class="header-
link">Register</a>
            </div>
        </div>
        <div class="content">
            <h1>Login</h1>
            <form method="POST" class="login-form">
                {% csrf_token %}

                <div class="form-group">
                    <label for="username">Username:</label>
                    <input type="text" name="username" id="username"
                           placeholder="Enter username" value="{{ username
}}">
                    {% if errors.username %}
                        <div class="field-error">{{ errors.username }}</div>
                    {% endif %}
                </div>

                <div class="form-group">
                    <label for="password">Password:</label>
                    <input type="password" name="password" id="password"
                           placeholder="Enter password">
                    {% if errors.password %}
                        <div class="field-error">{{ errors.password }}</div>
                    {% endif %}
                </div>

                {% if errors.invalid %}
                    <div class="error">{{ errors.invalid }}</div>
                {% endif %}

                <div class="form-group">
                    <input type="submit" value="Login">
                </div>
            </form>

            <p class="back-link">Don't have an account? <a href="{% url
'register' %}">Register here</a></p>
            <p class="back-link"><a href="{% url 'archive' %}">Back to
archive</a></p>
        </div>
    </body>
</html>
```

также добавим ссылки в archive.html и article.html:

```
        <div class="header-links">
            {% if user.is_authenticated %}
                <span class="username">{{ user.username }}</span>
                <a href="{% url 'create_post' %}" class="header-link">New
post</a>
                <a href="#" class="header-link">Logout</a>
            {% else %}
                <a href="{% url 'register' %}" class="header-
link">Register</a>
                <a href="{% url 'login' %}" class="header-link">Login</a>
            {% endif %}
        </div>
```

и стили для формы авторизации в article.css:

```
.login-form {
    max-width: 500px;
    margin: 0 auto;
}

.field-error {
    color: #ffcccc;
    font-size: 14px;
    margin-top: 5px;
    padding: 5px;
    background-color: rgba(255, 0, 0, 0.1);
    border-radius: 4px;
}
```

Проверим результат:

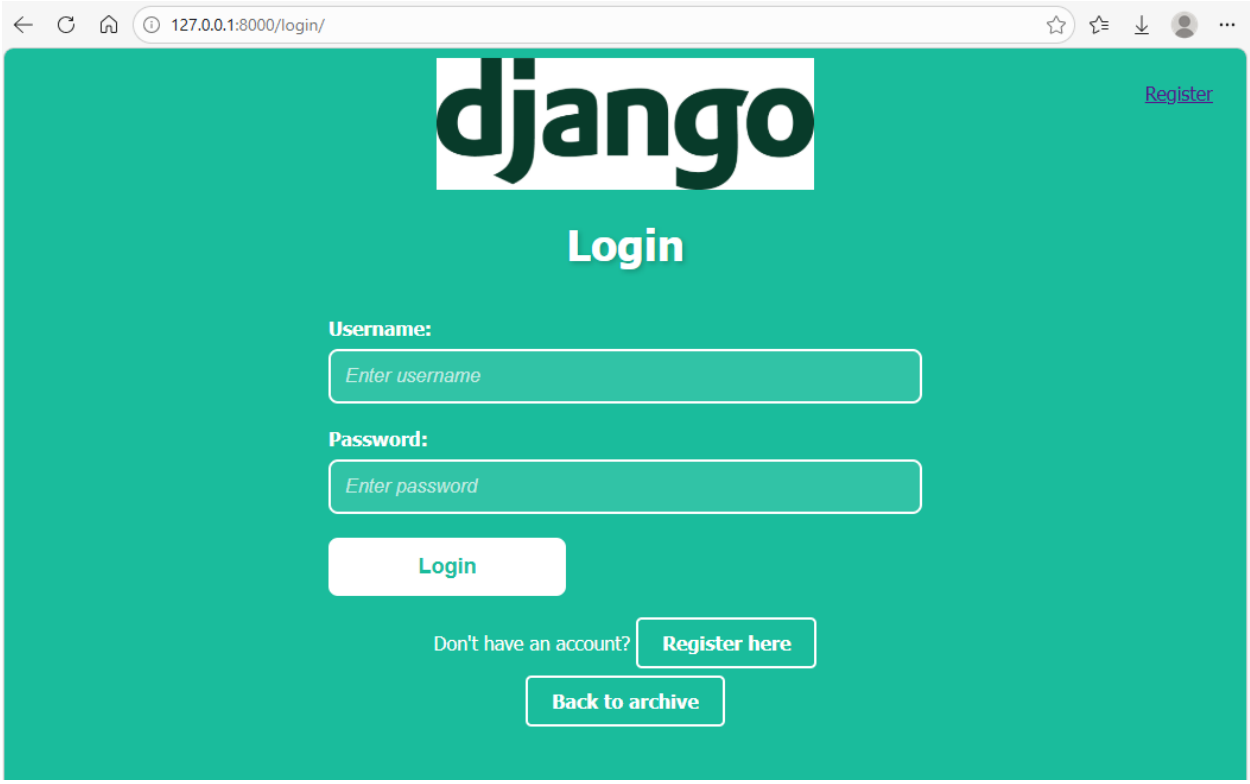

Рисунок 2 – Форма для авторизации пользователей

Лабораторная работа №7.

1. Изучение основ JavaScript, создание простейших функций и использование базовых операторов

Создадим папку js в articles/static/, а внутри нее файл helloworld.js. Затем подключим файл к веб-странице:

```
        </div>
        <script src="{% static 'js/helloworld.js' %}"></script>
    </body>
</html>
```

затем перейдем по адресу http://127.0.0.1:8000/ и запустим панель отладчика.

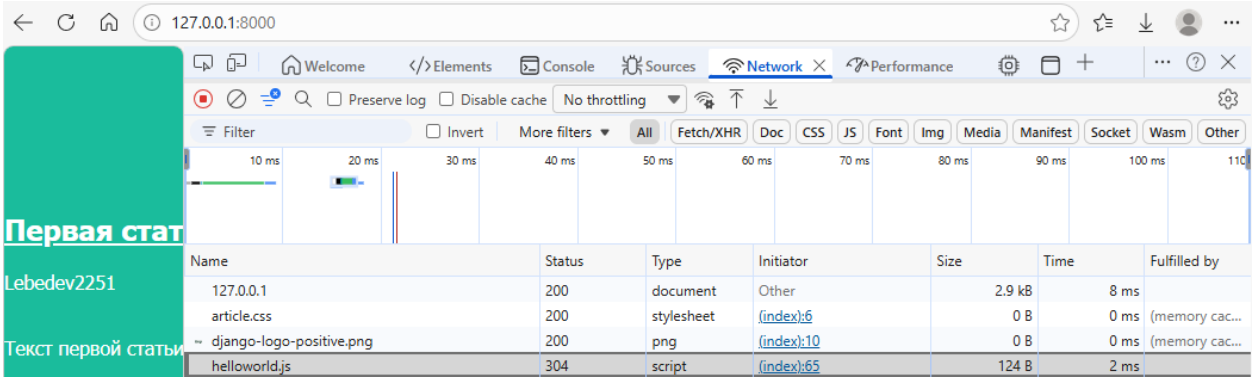

Рисунок 1 – Проверка наличия файла через панель отладчика

Отредактируем файл helloworld.js, добавив в него список студентов из первой лабораторной работы, обновим страницу браузера и проверим вывод в консоль:

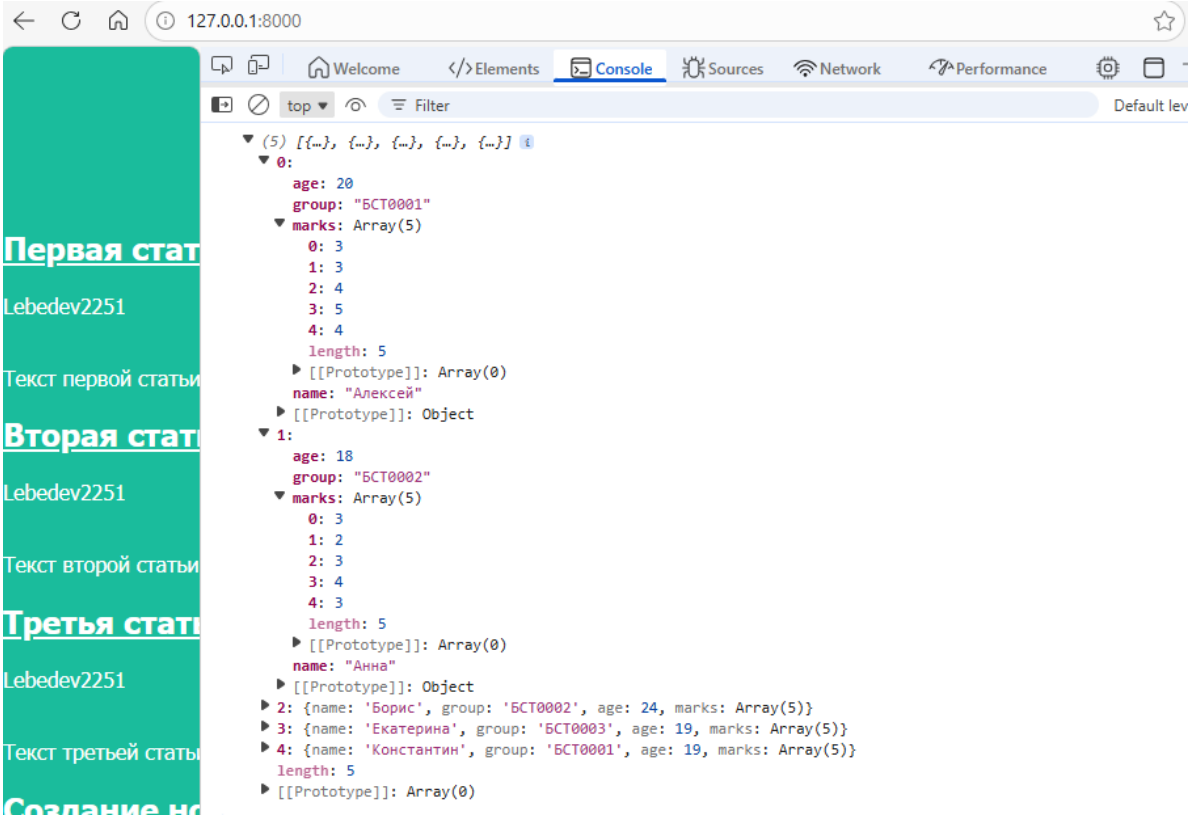

Рисунок 2 – Вывод списка студентов в консоль

Создадим функцию, которая будет выводить в виде таблицы содержимое массива groupmates:

```
// Функция для дополнения строки пробелами справа
var rpad = function(str, length) {
    str = str.toString();
    while (str.length < length)
        str = str + ' ';
    return str;
};

// Функция для вывода студентов в виде таблицы
var printStudents = function(students) {
    console.log(
        rpad("Имя студента", 15),
        rpad("Группа", 8),
        rpad("Возраст", 8),
        rpad("Оценки", 20)
    );

    for (var i = 0; i < students.length; i++) {
        console.log(
            rpad(students[i]['name'], 15),
```

```
            rpad(students[i]['group'], 8),
            rpad(students[i]['age'], 8),
            rpad(students[i]['marks'].join(', '), 20)
        );
    }
    console.log('\n');
};

// Функция фильтрации по среднему баллу
function filterStudentsByAvgMark(minAvgMark) {
    return groupmates.filter(student => {
        let sum = student.marks.reduce((a, b) => a + b, 0);
        let avg = sum / student.marks.length;
        return avg >= minAvgMark;
    });
}

console.log("Студенты со средним баллом >= 4.0");
let highAchievers = filterStudentsByAvgMark(4.0);
printStudents(highAchievers);
```

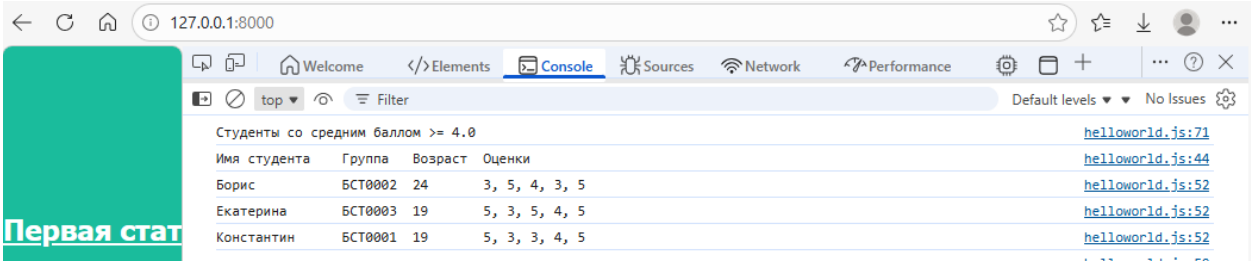

Рисунок 3 – Функция вывода студентов в виде таблицы по среднему баллу

Напишем функцию, которая фильтрует студентов по группе и должна возвращать только тех студентов, что учатся в указанной группе:

```
function filterStudentsByGroup(groupName) {
    return groupmates.filter(student => student.group === groupName);
}

console.log("Группа БСТ0001");
var group1Students = filterStudentsByGroup("БСТ0001");
printStudents(group1Students);

console.log("Группа  БСТ0002");
var group2Students = filterStudentsByGroup("БСТ0002");
printStudents(group2Students);
```

```
console.log("Группа  БСТ0003");
var group3Students = filterStudentsByGroup("БСТ0003");
printStudents(group3Students);
```

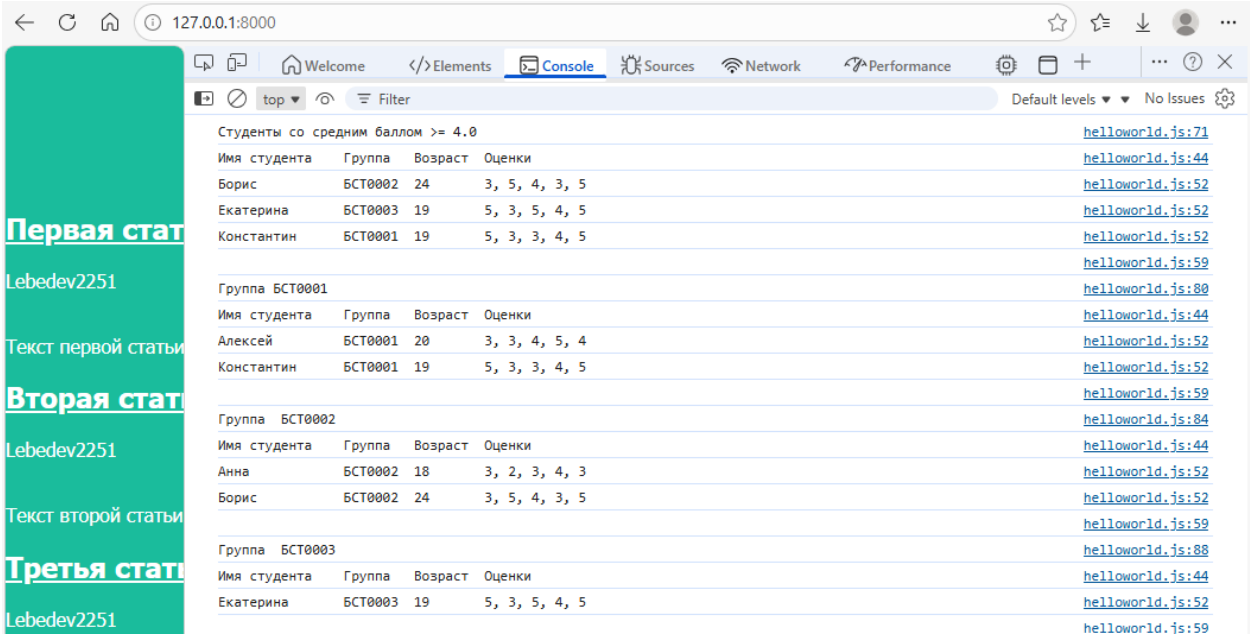

Рисунок 4 – Вывод в консоль функции фильтрации студентов по группе

2. Работа с элементами DOM с помощью JavaScript.

Добавим клавишу свернуть в archive.html для каждой статьи**:**

```
<button class="fold-button">свернуть</button>
```

и ссылку на сам скрипт:

```
<script src="{% static 'js/fold-post.js' %}"></script>
```

Создадим файл fold-post.js:

```
var foldBtns = document.getElementsByClassName("fold-button");

// Добавляем обработчик события для каждой кнопки
for (var i = 0; i < foldBtns.length; i++) {
    foldBtns[i].addEventListener("click", function(e) {
        // Проверяем, есть ли у кнопки класс folded
        if (e.target.className == "fold-button folded") {
            // Разворачиваем статью
            e.target.innerHTML = "свернуть";
            e.target.className = "fold-button";
```

```
            var displayState = "block";
        } else {
            // Сворачиваем статью
            e.target.innerHTML = "развернуть";
            e.target.className = "fold-button folded";
            var displayState = "none";
        }

        // Скрываем или показываем элементы статьи
        e.target
            .parentElement
            .getElementsByClassName('article-author')[0]
            .style.display = displayState;

        e.target
            .parentElement
            .getElementsByClassName('article-created-date')[0]
            .style.display = displayState;

        e.target
            .parentElement
            .getElementsByClassName('article-text')[0]
            .style.display = displayState;
    });
}
```

Добавим стили в article.css:

```
.fold-button {
    background-color: #ffffff;
    color: #1abc9c;
    border: 2px solid #ffffff;
    border-radius: 5px;
    padding: 5px 15px;
    font-weight: bold;
    cursor: pointer;
    margin-top: 10px;
    transition: all 0.3s ease;
}

.fold-button:hover {
    background-color: transparent;
    color: #ffffff;
}

.fold-button.folded {
    background-color: transparent;
```

```
    color: #ffffff;
}

.fold-button.folded:hover {
    background-color: #ffffff;
    color: #1abc9c;
}

.one-post {
    margin-bottom: 30px;
    padding: 20px;
    border: 2px solid rgba(255, 255, 255, 0.3);
    border-radius: 10px;
}
```

Проведем отладку:

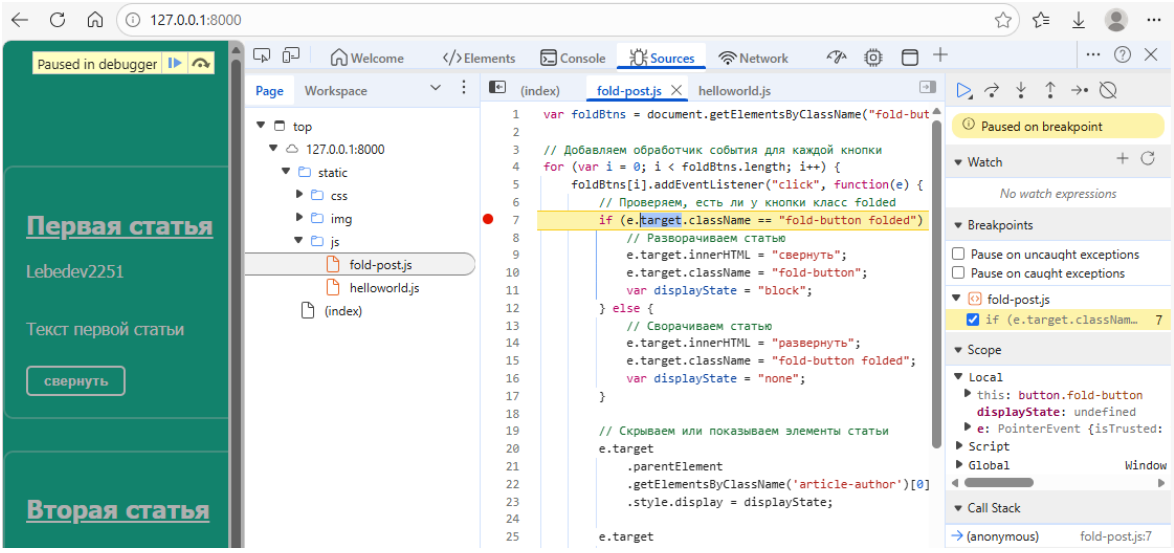

Рисунок 5 – Использование точки останова

Исследуем значения переменной event:

```
(index)   fold-post.js   helloworld.js

1  var foldBtns = document.getElementsByClassName("fold-but
2
3  // Добавляем обработчик события для каждой кнопки
4  for (var i = 0; i < foldBtns.length; i++) {
5      foldBtns[i].addEventListener("click", function(e) {
6          // Проверяем, есть ли у кнопки класс folded
7          if (e.target.className == "fold-button folded")
8              // Разворачиваем статью
9              e.target.innerHTML = "свернуть";
10             e.target.className = "fold-button";
11             var displayState = "block";
12         } else {
13             // Сворачиваем статью
14             e.target.innerHTML = "развернуть";
15             e.target.className = "fold-button folded";
16             var displayState = "none";
17         }
18
19         // Скрываем или показываем элементы статьи
20         e.target
21             .parentElement
22             .getElementsByClassName('article-author')[0]
23             .style.display = displayState;
24
25         e.target
26             .parentElement
27             .getElementsByClassName('article-created-dat
28             .style.display = displayState;
29
30         e.target
31             .parentElement
32             .getElementsByClassName('article-text')[0]
33             .style.display = displayState;
34     });
35 }
```

```
Local
  this: button.fold-button
  displayState: undefined
  e: PointerEvent
    isTrusted: true
    altitudeAngle: 1.5707963
    bubbles: true
    cancelable: true
    clientX: 67
    clientY: 322
    composed: true
    currentTarget: button.fo
    detail: 1
    eventPhase: 2
    height: 1
    layerX: 67
    layerY: 322
    offsetX: 43
    offsetY: 10
    pageX: 67
    pageY: 322
    pointerId: 1
    pointerType: "mouse"
    returnValue: true
    screenX: 861
    screenY: 403
    sourceCapabilities: Inpu
    srcElement: button.fold-
    target: button.fold-butt
    timeStamp: 818909.200000
    type: "click"
    view: Window {window: Wi
    which: 1
    width: 1
    x: 67
    y: 322
```

Рисунок 6 – Значения переменной e: PointerEvent

isTrusted: true - Событие вызвано реальным действием пользователя;

bubbles: true - свойство объекта конфигурации события, которое заставляет событие всплывать вверх по DOM-дереву от целевого элемента к его родителям;

clientX: 67 - координата X относительно окна браузера;

clientY: 322 - координата Y относительно окна браузера;

screenX: 861 - координата X относительно экрана;

screenY: 403 - координата Y относительно экрана;

target: button.fold-button - элемент, на который кликнули;

currentTarget: button.fold-button - текущий элемент обработчика;

pointerType: "mouse" - клик выполнен мышью и др.

Полученная страница с кнопкой «свернуть» / «развернуть» статью:

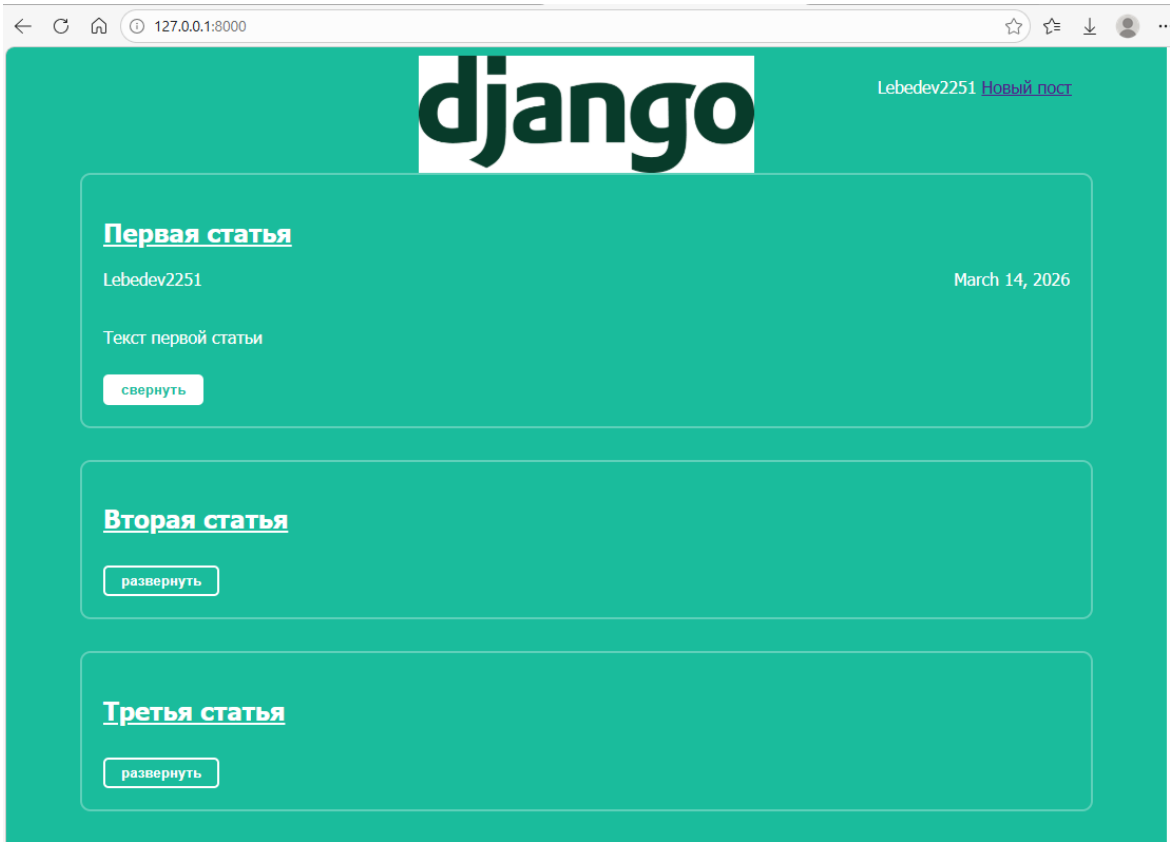

Рисунок 7 – Полученная страница

Сделаем более компактную реализацию функции-обработчика, для этого перепишем реализацию fold-post.js:

```
var foldBtns = document.getElementsByClassName("fold-button");

for (var i = 0; i < foldBtns.length; i++) {
    foldBtns[i].addEventListener("click", function(e) {
        var post = e.target.closest('.one-post');

        post.classList.toggle('folded');

        if (post.classList.contains('folded')) {
            e.target.innerHTML = "развернуть";
        } else {
```

```
            e.target.innerHTML = "свернуть";
        }
    });
}
```

через css-стили установим исчезновение остальных элементов поста:

```
.one-post.folded .article-author,
.one-post.folded .article-created-date,
.one-post.folded .article-text {
    display: none;
}

.one-post.folded .fold-button {
    background-color: transparent;
    color: #ffffff;
}

.one-post.folded .fold-button:hover {
    background-color: #ffffff;
    color: #1abc9c;
}

.fold-button {
    background-color: #ffffff;
    color: #1abc9c;
    border: 2px solid #ffffff;
    border-radius: 5px;
    padding: 5px 15px;
    font-weight: bold;
    cursor: pointer;
    margin-top: 10px;
    transition: all 0.3s ease;
}

.fold-button:hover {
    background-color: transparent;
    color: #ffffff;
}
```

Лабораторная работа №8.

1. Изучение библиотеки jQuery, добавление подсветки для наведенного поста и эффекта для картинки-логотипа.

Подключим библиотеку и скрипт к странице со списком всех записей в archive.html:

```
        <script src="{% static 'js/jquery-4.0.0.min.js' %}"></script>
        <script src="{% static 'js/helloworld.js' %}"></script>
        <script src="{% static 'js/fold-post.js' %}"></script>
        <script src="{% static 'js/highlight-post.js' %}"></script>
    </body>
```

Создадим файл highlight-post.js:

```
$(document).ready(function() {
    console.log("Highlight post script loaded");

    $('.one-post').hover(
        function() {
            // Mouse enter
            console.log("Mouse entered post");
        },
        function() {
            // Mouse leave
            console.log("Mouse left post");
        }
    );
});
```

добавим стили для теней в article.css:

```
.one-post {
    position: relative;
    margin-bottom: 30px;
    padding: 20px;
    border: 2px solid rgba(255, 255, 255, 0.3);
    border-radius: 10px;
    overflow: hidden;
}

.one-post-shadow {
    position: absolute;
    top: 0;
    left: 0;
    height: 100%;
```

```
    width: 100%;
    background: black;
    opacity: 0;
    z-index: -1;
    transition: opacity 0.3s ease;
}

.one-post:hover .one-post-shadow {
    opacity: 0.2;
}
```

Проверка результата:

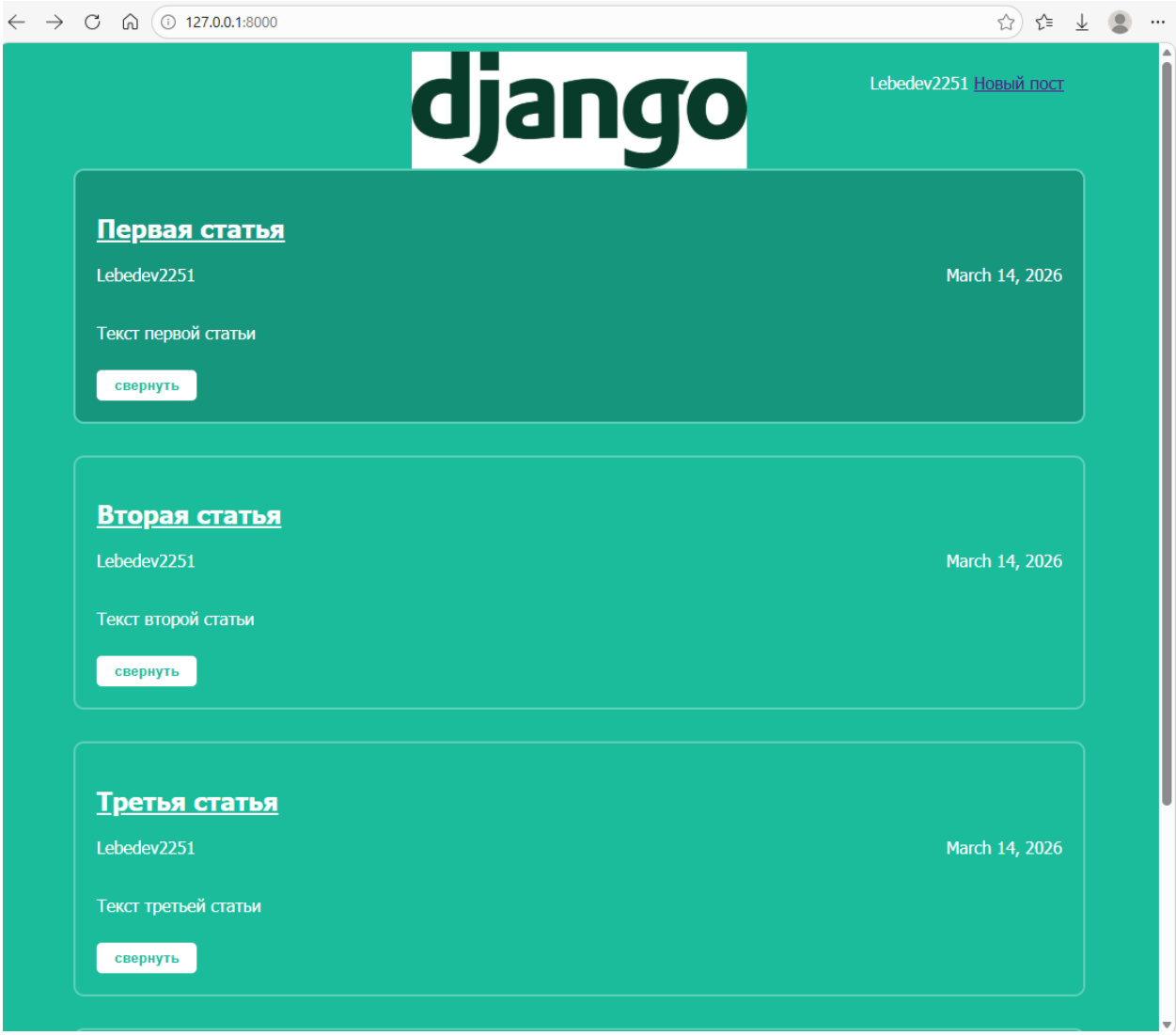

Рисунок 1 – Работа затемнения при наведение на пост

Сделаем увеличение логотипа при наведении на него курсора. Для этого добавим в highlight-post.js:

```
    // Эффект для логотипа
    $('.header img').hover(
        function() {
            $(this).stop().animate({
                width: '338px'
            }, 200);
        },
        function() {
            $(this).stop().animate({
                width: '318px'
            }, 200);
        }
    );
});
```

Лабораторная работа №9.

1. Дальнейшее изучение библиотеки jQuery, добавление эффекта параллакса для главной страницы блога.

Изменим разметку в файле archive.html:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Архив всех статей</title>
        {% load static %}
        <link rel="stylesheet" href="{% static 'css/article.css' %}">
    </head>
    <body class="archive">
        <div class="header">
            <img class="logo" src="{% static 'img/django-logo-positive.png'
%}" alt="Логотип"/>
            <div class="header-text-area">
                <p class="service-title">Коллективный блог</p>
                <div class="auth-links">
                    {% if user.is_authenticated %}
                        <span class="username">{{ user.username }}</span>
                        <a href="{% url 'create_post' %}" class="register-
link">Новый пост</a>
                    {% else %}
                        <a href="{% url 'register' %}" class="register-
link">Регистрация</a>
                        <a href="{% url 'login' %}" class="register-
link">Вход</a>
                    {% endif %}
                </div>
            </div>
            <div class="icons-for-parallax">
                <img class="icon-for-parallax-first" src="{% static
'img/django-logo-positive.png' %}">
                <img class="icon-for-parallax-second" src="{% static
'img/django-logo-positive.png' %}">
                <img class="icon-for-parallax-third" src="{% static
'img/django-logo-positive.png' %}">
            </div>
        </div>

        <div class="archive">
            {% for post in posts %}
            <div class="one-post">
                <h2 class="post-title">
```

```
                    <a href="{% url 'get_article' article_id=post.id %}">{{
post.title }}</a>
                </h2>
                <div class="article-info">
                    <div class="article-author">{{ post.author.username
}}</div>
                    <div class="article-created-date">{{ post.created_date
}}</div>
                </div>
                <p class="article-text">{{ post.get_excerpt }}</p>
                <button class="fold-button">свернуть</button>
                <div class="one-post-shadow"></div>
            </div>
            {% endfor %}
        </div>

        <script src="{% static 'js/jquery-4.0.0.min.js' %}"></script>
        <script src="{% static 'js/helloworld.js' %}"></script>
        <script src="{% static 'js/fold-post.js' %}"></script>
        <script src="{% static 'js/highlight-post.js' %}"></script>
        <script src="{% static 'js/parallax.js' %}"></script>
    </body>
</html>
```

создадим файл parallax.js:

```
$(document).ready(function(){
    var yPosition;
    var scrolled = 0;
    var $parallaxElements = $('.icons-for-parallax img');

    console.log("Parallax script loaded");
    console.log("Parallax elements found:", $parallaxElements.length);

    $(window).scroll(function() {
        scrolled = $(window).scrollTop();

        for (var i = 0; i < $parallaxElements.length; i++) {
            yPosition = (scrolled * 0.15 * (i + 1));

            $parallaxElements.eq(i).css({
                top: yPosition + 'px'
            });
        }
    });
});
```

изменим стили в article.css:

```
body.archive {
    background: #f5f5f5;
    color: #333333;
}

.header {
    background: #2c3e50;
    padding: 20px 40px;
    position: relative;
    overflow: hidden;
    min-height: 300px;
    display: flex;
    align-items: flex-start;
    justify-content: space-between;
}

.header .logo {
    width: 150px;
    height: auto;
    margin-right: 30px;
}

.header-text-area {
    float: left;
    width: 400px;
    color: #ffffff;
}

.service-title {
    font-size: 32px;
    font-weight: bold;
    margin: 0 0 20px 0;
    color: #ffffff;
}

.auth-links {
    margin-top: 10px;
}

.auth-links .register-link {
    color: #ffffff;
    text-decoration: none;
    margin-right: 15px;
    padding: 8px 16px;
    border: 2px solid #ffffff;
    border-radius: 5px;
```

```
    transition: all 0.3s ease;
}

.auth-links .register-link:hover {
    background-color: #ffffff;
    color: #2c3e50;
}

.auth-links .username {
    color: #ffffff;
    font-weight: bold;
    margin-right: 15px;
}

.icons-for-parallax {
    position: relative;
    height: 300px;
    width: 400px;
    float: right;
}

.icons-for-parallax img {
    position: absolute;
    top: 0;
    left: 0;
    transition: all 0.1s linear;
}

.icon-for-parallax-first {
    width: 200px;
    z-index: 3;
    margin-top: 50px;
}

.icon-for-parallax-second {
    width: 175px;
    z-index: 2;
    margin-top: 100px;
    margin-left: 80px;
}

.icon-for-parallax-third {
    width: 150px;
    z-index: 1;
    margin-top: 150px;
    margin-left: 160px;
}
```

```
.archive {
    width: 960px;
    margin: 40px auto;
    background: transparent;
}

.post-title a {
    color: #2c3e50;
}

.article-author, .article-created-date {
    color: #7f8c8d;
}
```

Проверим полученный эффект:

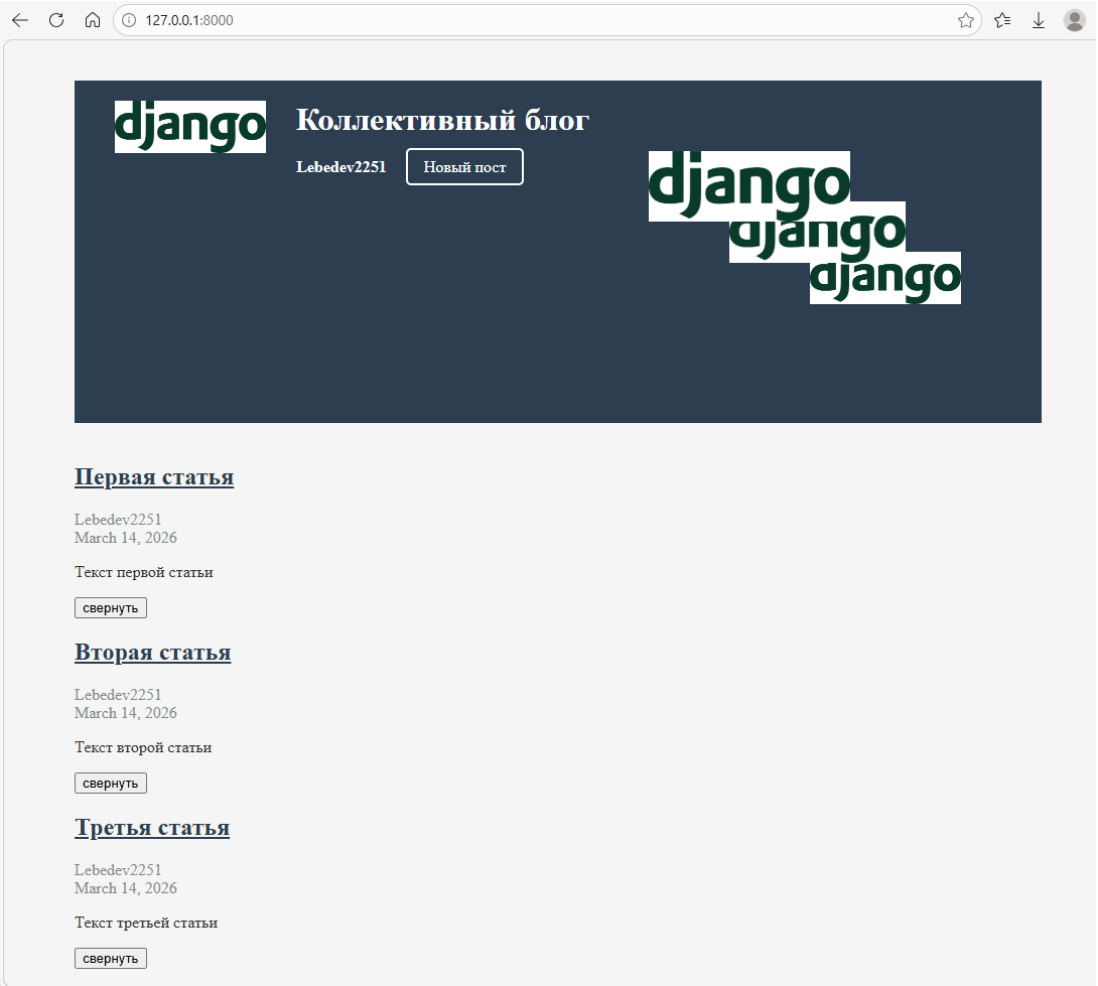

Рисунок 1 – Главная страница с эффектом параллакса

Настроим также эффект параллакса для картинки-логотипа. Для этого добавим следующие изменения в parallax.js:

```
$(document).ready(function(){
    var yPosition;
    var scrolled = 0;

    var $parallaxElements = $('.icons-for-parallax img');
    var $logo = $('.header .logo');

    console.log("Parallax script loaded");
    console.log("Parallax elements found:", $parallaxElements.length);
    console.log("Logo found:", $logo.length);

    $(window).scroll(function() {
        scrolled = $(window).scrollTop();

        for (var i = 0; i < $parallaxElements.length; i++) {
            yPosition = (scrolled * 0.15 * (i + 1));
            $parallaxElements.eq(i).css({
                top: yPosition + 'px'
            });
        }

        var logoPosition = scrolled * 0.3;
        $logo.css({
            top: logoPosition + 'px',
            position: 'relative'
        });
    });
});
```

Список литературы

1. Medvedev, M. A. Development Web Applications with Django Framework : textbook / M. A. Medvedev, M. A. Medvedeva ; под редакцией D. B. Berg. — Екатеринбург : Издательство Уральского университета, 2024. — 140 с.
2. Меле, А. Django 4 в примерах / А. Меле. — Москва : ДМК Пресс, 2023. — 800 с.
3. Django documentation [Электронный ресурс] // Django Software Foundation. — Режим доступа: https://docs.djangoproject.com/, свободный. — (дата обращения: 07.03.2026)